

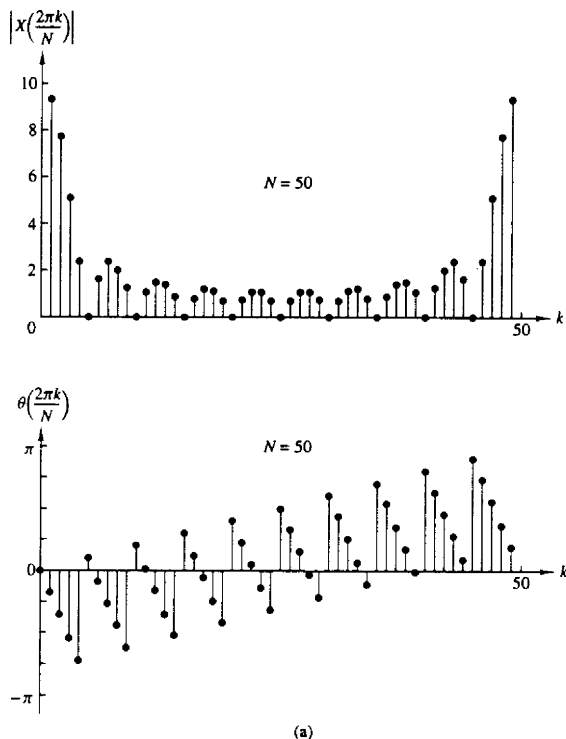


Figure 56 Magnitude and phase of an N -point DFT in Example 6.4.2; (a) $L = 10$, $N = 50$; (b) $L = 10$, $N = 100$.

samples, and an $N \times N$ matrix $\mathbf{W}_N$ as

$$\mathbf{x}_N = \begin{bmatrix} x(0) \\ x(1) \\ \vdots \\ x(N-1) \end{bmatrix}, \quad \mathbf{X}_N = \begin{bmatrix} X(0) \\ X(1) \\ \vdots \\ X(N-1) \end{bmatrix}$$

$$\mathbf{W}_N = \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & W_N & W_N^2 & \dots & W_N^{N-1} \\ & W_N^2 & W_N^4 & \dots & W_N^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & W_N^{N-1} & W_N^{2(N-1)} & \dots & W_N^{(N-1)(N-1)} \end{bmatrix} \quad (5.1.23)$$

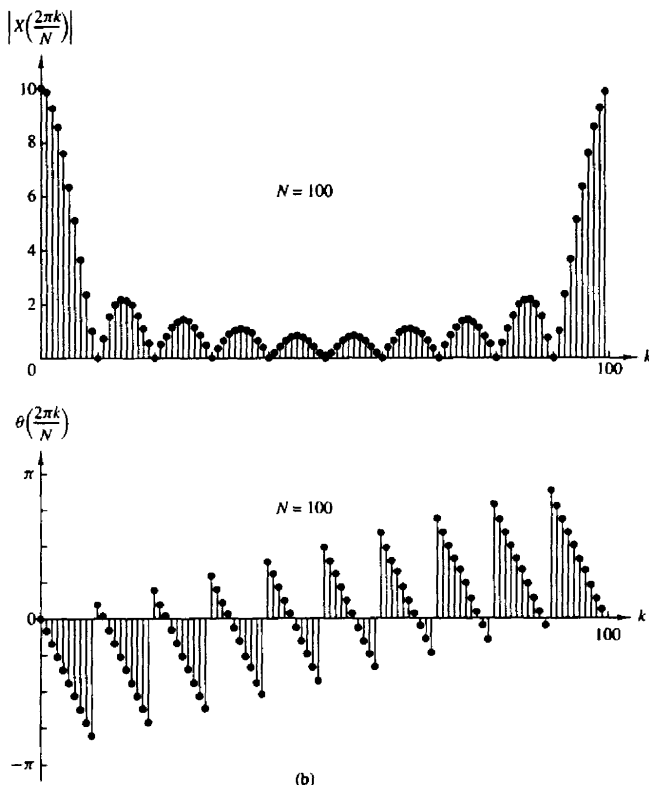


Figure 56 continued

With these definitions, the N -point DFT may be expressed in matrix form as

$$\mathbf{X}_N = \mathbf{W}_N \mathbf{x}_N \quad (5.1.24)$$

where $\mathbf{W}_N$ is the matrix of the linear transformation. We observe that $\mathbf{W}_N$ is a symmetric matrix. If we assume that the inverse of $\mathbf{W}_N$ exists, then (5.1.24) can be inverted by premultiplying both sides by $\mathbf{W}_N^{-1}$. Thus we obtain

$$\mathbf{x}_N = \mathbf{W}_N^{-1} \mathbf{X}_N \quad (5.1.25)$$

But this is just an expression for the IDFT.

In fact, the IDFT as given by (5.1.21), can be expressed in matrix form as

$$\mathbf{x}_N = \frac{1}{N} \mathbf{W}_N^* \mathbf{X}_N \quad (5.1.26)$$

where $\mathbf{W}_N^*$ denotes the complex conjugate of the matrix $\mathbf{W}_N$. comparison of (5.1.26) with (5.1.25) leads us to conclude that

$$\mathbf{W}_N^{-1} = \frac{1}{N} \mathbf{W}_N^* \quad (5.1.27)$$

which, in turn, implies that

$$\mathbf{W}_N \mathbf{W}_N^* = N \mathbf{I}_N \quad (5.1.28)$$

where $\mathbf{I}_N$ is an $N \times N$ identity matrix. Therefore, the matrix $\mathbf{W}_N$ in the transformation is an orthogonal (unitary) matrix. Furthermore, its inverse exists and is given as $\mathbf{W}_N^*/N$. Of course, the existence of the inverse of $\mathbf{W}_N$ was established previously from our derivation of the IDFT.

Example 5. U

Compute the DFT of the four-point sequence

$$x(n) = (0 \quad 1 \quad 2 \quad 3)$$

Solution The first step is to determine the matrix $\mathbf{W}_4$. By exploiting the periodicity property of $\mathbf{W}_4$ and the symmetry property

$$\mathbf{W}_N^{k+N/2} = -\mathbf{W}_N^k$$

the matrix $\mathbf{W}_4$ may be expressed as

$$\begin{aligned} \mathbf{W}_4 &= \begin{bmatrix} W_4^0 & W_4^0 & W_4^0 & W_4^0 \\ W_4^0 & W_4^1 & W_4^2 & W_4^3 \\ W_4^0 & W_4^2 & W_4^4 & W_4^6 \\ W_4^0 & W_4^3 & W_4^6 & W_4^9 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & W_4^1 & W_4^2 & W_4^3 \\ 1 & W_4^2 & W_4^4 & W_4^6 \\ 1 & W_4^3 & W_4^6 & W_4^9 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix} \end{aligned}$$

Then

$$\mathbf{X}_4 = \mathbf{W}_4 \mathbf{x}_4 = \begin{bmatrix} 6 \\ -2 + 2j \\ -2 \\ -2 - 2j \end{bmatrix}$$

The IDFT of $\mathbf{X}_4$ may be determined by conjugating the elements in $\mathbf{W}_4$ to obtain $\mathbf{W}_4^*$ and then applying the formula (5.1.26).

The DFT and IDFT are computational tools that play a very important role in many digital signal processing applications, such as frequency analysis (spectrum analysis) of signals, power spectrum estimation, and linear filtering. The importance of the DFT and IDFT in such practical applications is due to a large extent on the existence of computationally efficient algorithms, known collectively as fast

Fourier transform (FFT) algorithms, for computing the DFT and IDFT. This class of algorithms is described in Chapter 6.

5.1.4 Relationship of the DFT to Other Transforms

In this discussion we have indicated that the DFT is an important computational tool for performing frequency analysis of signals on digital signal processors. In view of the other frequency analysis tools and **transforms** that we have developed, it is important to establish the relationships between the DFT to these other **transforms**.

Relationship to the Fourier series coefficients of a periodic sequence.

A periodic sequence $\{x_p(n)\}$ with fundamental period N can be represented in a Fourier series of the **form**

$$x_p(n) = \sum_{k=0}^{N-1} c_k e^{j2\pi nk/N} \quad -\infty < n < \infty \quad (5.1.29)$$

where the Fourier series coefficients are given by the expression

$$c_k = \frac{1}{N} \sum_{n=0}^{N-1} x_p(n) e^{-j2\pi nk/N} \quad k = 0, 1, \dots, N-1 \quad (5.1.30)$$

If we compare (5.1.29) and (5.1.30) with (5.1.18) and (5.1.19), we observe that the formula for the Fourier series coefficients has the **form** of a DFT. In fact, if we define a sequence $x(n) = x_p(n)$, $0 \leq n \leq N-1$, the **DFT** of this sequence is simply

$$X(k) = N c_k \quad (5.1.31)$$

Furthermore, (5.1.29) has the form of an IDFT. **Thus** the N -point DFT provides the exact line spectrum of a periodic sequence with fundamental period N .

Relationship to the Fourier transform of an aperiodic sequence.

We have already shown that if $x(n)$ is an aperiodic finite energy sequence with Fourier transform $X(\omega)$, which is sampled at N equally spaced frequencies $\omega_k = 2\pi k/N$, $k = 0, 1, \dots, N-1$, the spectral components

$$X(k) = X(\omega)|_{\omega=2\pi k/N} = \sum_{n=-\infty}^{\infty} x(n) e^{-j2\pi nk/N} \quad k = 0, 1, \dots, N-1 \quad (5.1.32)$$

are the DFT coefficients of the periodic sequence of period N , given by

$$x_p(n) = \sum_{l=-\infty}^{\infty} x(n-lN) \quad (5.1.33)$$

Thus $x_p(n)$ is determined by aliasing $\{x(n)\}$ over the interval $0 \leq n \leq N-1$. The finite-duration sequence

$$\hat{x}(n) = \begin{cases} x_p(n), & 0 \leq n \leq N-1 \\ 0, & \text{otherwise} \end{cases} \quad (5.1.34)$$

bears no resemblance to the **original** sequence $\{x(n)\}$, unless $x(n)$ is of finite duration and length $L \leq N$, in which case

$$x(n) = \hat{x}(n) \quad 0 \leq n \leq N-1 \quad (5.1.35)$$

Only in this case will the IDFT of $\{X(k)\}$ yield the original sequence $\{x(n)\}$.

Relationship to the z-transform. Let us consider a sequence $x(n)$ having the z-transform

$$X(z) = \sum_{n=-\infty}^{\infty} x(n)z^{-n} \quad (5.1.36)$$

with a ROC that includes the unit circle. If $X(z)$ is sampled at the N equally spaced points on the unit circle $z_k = e^{j2\pi k/N}$, $0, 1, 2, \dots, N-1$, we obtain

$$\begin{aligned} X(k) &\equiv X(z)|_{z=e^{j2\pi k/N}} \quad k = 0, 1, \dots, N-1 \\ &= \sum_{n=-\infty}^{\infty} x(n)e^{-j2\pi nk/N} \end{aligned} \quad (5.1.37)$$

The expression in (5.1.37) is identical to the Fourier transform $X(\omega)$ evaluated at the N equally spaced frequencies $\omega_k = 2\pi k/N$, $k = 0, 1, \dots, N-1$, which is the topic treated in Section 5.1.1.

If the sequence $x(n)$ has a finite duration of length N or less, the sequence can be recovered from its N -point DFT. Hence its z-transform is uniquely **determined** by its N -point DFT. Consequently, $X(z)$ can be expressed as a function of the DFT $\{X(k)\}$ as follows

$$\begin{aligned} X(z) &= \sum_{n=0}^{N-1} x(n)z^{-n} \\ X(z) &= \sum_{n=0}^{N-1} \left[\frac{1}{N} \sum_{k=0}^{N-1} X(k)e^{j2\pi kn/N} \right] z^{-n} \\ X(z) &= \frac{1}{N} \sum_{k=0}^{N-1} X(k) \sum_{n=0}^{N-1} (e^{j2\pi k/N} z^{-1})^n \\ X(z) &= \frac{1-z^{-N}}{N} \sum_{k=0}^{N-1} \frac{X(k)}{1-e^{j2\pi k/N} z^{-1}} \end{aligned} \quad (5.1.38)$$

When evaluated on the unit circle, (5.1.38) yields the Fourier transform of the finite-duration sequence in terms of its DFT, in the form

$$X(\omega) = \frac{1-e^{-j\omega N}}{N} \sum_{k=0}^{N-1} \frac{X(k)}{1-e^{-j(\omega-2\pi k/N)}} \quad (5.1.39)$$

This expression for the Fourier transform is a polynomial (Lagrange) interpolation formula for $X(\omega)$ expressed in terms of the **values** $\{X(k)\}$ of the **polynomial** at a set of equally spaced discrete frequencies $\omega_k = 2\pi k/N$, $k = 0, 1, \dots, N-1$. With

some algebraic manipulations, it is possible to reduce (5.1.39) to the interpolation formula given previously in (5.1.13).

Relationship to the Fourier series coefficients of a continuous-time signal. Suppose that $x_a(t)$ is a continuous-time periodic signal with fundamental period $T_p = 1/F_0$. The signal can be expressed in a Fourier series

$$x_a(t) = \sum_{k=-\infty}^{\infty} c_k e^{j2\pi k F_0 t} \quad (5.1.40)$$

where $\{c_k\}$ are the Fourier coefficients. If we sample $x_a(t)$ at a uniform rate $F_s = N/T_p = 1/T$, we obtain the discrete-time sequence

$$\begin{aligned} x(n) \equiv x_a(nT) &= \sum_{k=-\infty}^{\infty} c_k e^{j2\pi k F_0 nT} = \sum_{k=-\infty}^{\infty} c_k e^{j2\pi k n/N} \\ &= \sum_{l=0}^{N-1} \left[\sum_{k=lN}^{\infty} c_k \right] e^{j2\pi k n/N} \end{aligned} \quad (5.1.41)$$

It is clear that (5.1.41) is in the form of an IDFT formula, where

$$X(k) = N \sum_{l=-\infty}^{\infty} c_{k-lN} \equiv N \tilde{c}_k \quad (5.1.42)$$

and

$$\tilde{c}_k = \sum_{l=-\infty}^{\infty} c_{k-lN} \quad (5.1.43)$$

Thus the $\{\tilde{c}_k\}$ sequence is an aliased version of the sequence $\{c_k\}$.

5.2 PROPERTIES OF THE DFT

In Section 5.1.2 we introduced the **DFT** as a set of N samples $\{X(k)\}$ of the Fourier transform $X(\omega)$ for a finite-duration sequence $\{x(n)\}$ of length $L \leq N$. The sampling of $X(\omega)$ occurs at the N equally spaced frequencies $\omega_k = 2\pi k/N$, $k = 0, 1, 2, \dots, N-1$. We demonstrated that the N samples $\{X(k)\}$ uniquely represent the sequence $\{x(n)\}$ in the frequency domain. Recall that the DFT and inverse DFT (IDFT) for an N -point sequence $\{x(n)\}$ are given as

$$\text{DFT: } X(k) = \sum_{n=0}^{N-1} x(n) W_N^{kn} \quad k = 0, 1, \dots, N-1 \quad (5.2.1)$$

$$\text{IDFT: } x(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k) W_N^{-kn} \quad n = 0, 1, \dots, N-1 \quad (5.2.2)$$

where W_N is defined as

$$W_N = e^{-j2\pi/N} \quad (5.2.3)$$

In this section we present the important properties of the DFT. In view of the relationships established in Section 5.1.4 between the **DFT** and Fourier series, and Fourier **transforms** and **z-transforms** of discrete-time signals, we expect the properties of the DFT to resemble the properties of these other transforms and series. However, some important differences exist, one of which is the circular convolution property derived in the following section. A **good** understanding of these properties is extremely helpful in the application of the **DFT** to practical problems.

The notation used below to denote the N-point **DFT** pair $x(n)$ and $X(k)$ is

$$x(n) \xleftrightarrow[N]{\text{DFT}} X(k)$$

5.2.1 Periodicity, Linearity, and Symmetry Properties

Periodicity. If $x(n)$ and $X(k)$ are an N-point DFT pair, then

$$x(n + N) = x(n) \quad \text{for all } n \quad (5.2.4)$$

$$X(k + N) = X(k) \quad \text{for all } k \quad (5.2.5)$$

These periodicities in $x(n)$ and $X(k)$ follow immediately from formulas (5.2.1) and (5.2.2) for the **DFT** and **IDFT**, respectively.

We previously illustrated the periodicity property in the sequence $x(n)$ for a given DFT. However, we had not previously viewed the DFT $X(k)$ as a periodic sequence. In some applications it is advantageous to do this.

Linearity. If

$$x_1(n) \xleftrightarrow[N]{\text{DFT}} X_1(k)$$

and

$$x_2(n) \xleftrightarrow[N]{\text{DFT}} X_2(k)$$

then for any real-valued or complex-valued constants a_1 and a_2 ,

$$a_1 x_1(n) + a_2 x_2(n) \xleftrightarrow[N]{\text{DFT}} a_1 X_1(k) + a_2 X_2(k) \quad (5.2.6)$$

This property follows immediately from the definition of the **DFT** given by (5.2.1).

Circular Symmetries of a Sequence. As we have seen, the N-point **DFT** of a finite duration sequence, $x(n)$ of length $L \leq N$ is equivalent to the N-point **DFT** of a periodic sequence $x_p(n)$, of period N, which is obtained by periodically extending $x(n)$, that is,

$$x_p(n) = \sum_{l=-\infty}^{\infty} x(n - lN) \quad (5.2.7)$$

Now suppose that we shift the periodic sequence $x_p(n)$ by k units to the right. Thus we obtain another periodic sequence

$$x'_p(n) = x_p(n - k) = \sum_{l=-\infty}^{\infty} x(n - k - lN) \quad (5.2.8)$$

The finite-duration sequence

$$x'(n) = \begin{cases} x'_p(n), & 0 \leq n \leq N-1 \\ 0, & \text{otherwise} \end{cases} \quad (5.2.9)$$

is related to the original sequence $x(n)$ by a circular shift. This relationship is illustrated in Fig. 5.7 for $N = 4$.

In general, the circular shift of the sequence can be represented as the index modulo N . Thus we can write

$$\begin{aligned} x'(n) &= x(n - k, \text{ modulo } N) \\ &\equiv x((n - k))_N \end{aligned} \quad (5.2.10)$$

For example, if $k = 2$ and $N = 4$, we have

$$x'(n) = x((n - 2))_4$$

which implies that

$$x'(0) = x((-2))_4 = x(2)$$

$$x'(1) = x((-1))_4 = x(3)$$

$$x'(2) = x((0))_4 = x(0)$$

$$x'(3) = x((1))_4 = x(1)$$

Hence $x'(n)$ is simply $x(n)$ shifted circularly by two units in time, where the counterclockwise direction has been arbitrarily selected as the positive direction. Thus we conclude that a circular shift of an N -point sequence is equivalent to a linear shift of its periodic extension, and *vice versa*.

The inherent periodicity resulting from the arrangement of the N -point sequence on the circumference of a circle dictates a different definition of even and odd symmetry, and time reversal of a sequence.

An N -point sequence is called circularly **even** if it is symmetric about the point zero on the circle. This implies that

$$x(N - n) = x(n) \quad 1 \leq n \leq N - 1 \quad (5.2.11)$$

An N -point sequence is called circularly **odd** if it is antisymmetric about the point zero on the circle. This implies that

$$x(N - n) = -x(n) \quad 1 \leq n \leq N - 1 \quad (5.2.12)$$

The time reversal of an N -point sequence is attained by reversing its samples about the point zero on the circle. **Thus** the sequence $x((-n))_N$ is simply given as

$$x((-n))_N = x(N - n) \quad 0 \leq n \leq N - 1 \quad (5.2.13)$$

This time reversal is equivalent to plotting $x(n)$ in a clockwise direction on a circle.

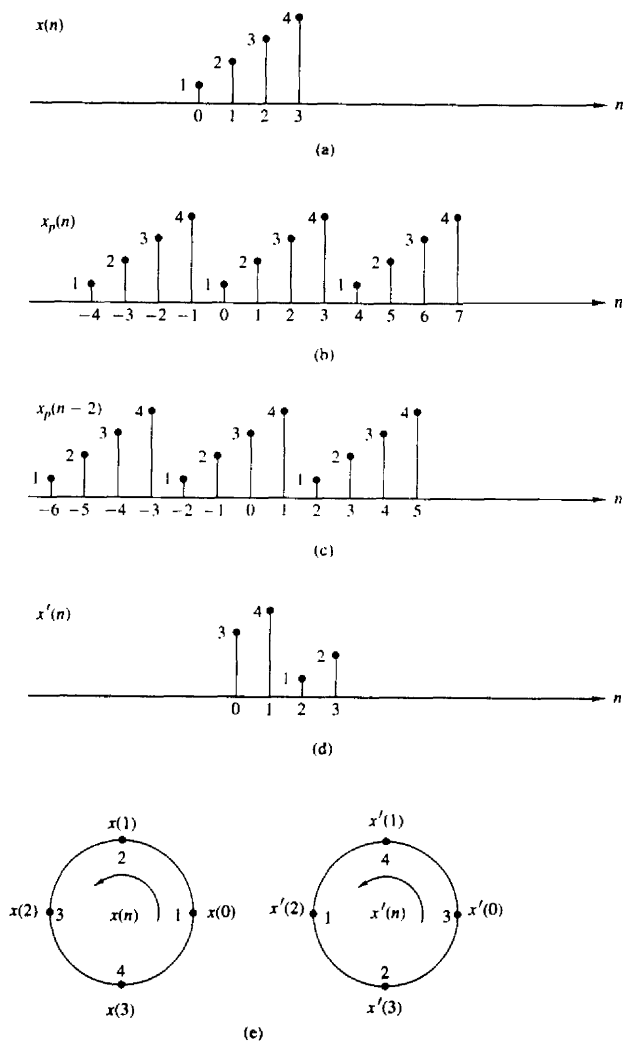


Figure 5.7 Circular shift of a sequence.

An equivalent definition of even and odd sequences for the associated periodic sequence $x_p(n)$ is given as follows

$$\begin{aligned}\text{even: } x_p(n) &= x_p(-n) = x_p(N-n) \\ \text{odd: } x_p(n) &= -x_p(-n) = -x_p(N-n)\end{aligned}\quad (5.2.14)$$

If the periodic sequence is complex-valued, we have

$$\begin{aligned}\text{conjugate even: } x_p(n) &= x_p^*(N-n) \\ \text{conjugate odd: } x_p(n) &= -x_p^*(N-n)\end{aligned}\quad (5.2.15)$$

These relationships suggest that we decompose the sequence $x_p(n)$ as

$$x_p(n) = x_{pe}(n) + x_{po}(n) \quad (5.2.16)$$

where

$$\begin{aligned}x_{pe}(n) &= \frac{1}{2}[x_p(n) + x_p^*(N-n)] \\ x_{po}(n) &= \frac{1}{2}[x_p(n) - x_p^*(N-n)]\end{aligned}\quad (5.2.17)$$

Symmetry properties of the DFT. The symmetry properties for the DFT can be obtained by applying the methodology previously used for the Fourier transform. Let us assume that the N -point sequence $x(n)$ and its DFT are both complex valued. Then the sequences can be expressed as

$$x(n) = x_R(n) + jx_I(n) \quad 0 \leq n \leq N-1 \quad (5.2.18)$$

$$X(k) = X_R(k) + jX_I(k) \quad 0 \leq k \leq N-1 \quad (5.2.19)$$

By substituting (5.2.18) into the expression for the DFT given by (5.2.1), we obtain

$$X_R(k) = \sum_{n=0}^{N-1} \left[x_R(n) \cos \frac{2\pi kn}{N} + x_I(n) \sin \frac{2\pi kn}{N} \right] \quad (5.2.20)$$

$$X_I(k) = - \sum_{n=0}^{N-1} \left[x_R(n) \sin \frac{2\pi kn}{N} - x_I(n) \cos \frac{2\pi kn}{N} \right] \quad (5.2.21)$$

Similarly, by substituting (5.2.19) into the expression for the IDFT given by (5.2.2), we obtain

$$x_R(n) = \frac{1}{N} \sum_{k=0}^{N-1} \left[X_R(k) \cos \frac{2\pi kn}{N} - X_I(k) \sin \frac{2\pi kn}{N} \right] \quad (5.2.22)$$

$$x_I(n) = \frac{1}{N} \sum_{k=0}^{N-1} \left[X_R(k) \sin \frac{2\pi kn}{N} + X_I(k) \cos \frac{2\pi kn}{N} \right] \quad (5.2.23)$$

Real-valued sequences. If the sequence $x(n)$ is real, it follows directly from (5.2.1) that

$$X(N-k) = X^*(k) = X(-k) \quad (5.2.24)$$

Consequently, $|X(N-k)| = |X(k)|$ and $\angle X(N-k) = -\angle X(k)$. Furthermore, $x_I(n) = 0$ and therefore $x(n)$ can be determined from (5.2.22), which is another form for the IDFT.

Real and even sequences. If $x(n)$ is real and even, that is,

$$x(n) = x(N-n) \quad 0 \leq n \leq N-1$$

then (5.2.21) yields $X_I(k) = 0$. Hence the DFT reduces to

$$X(k) = \sum_{n=0}^{N-1} x(n) \cos \frac{2\pi kn}{N} \quad 0 \leq k \leq N-1 \quad (5.2.25)$$

which is itself real-valued and even. Furthermore, since $X_I(k) = 0$, the IDFT reduces to

$$x(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k) \cos \frac{2\pi kn}{N} \quad 0 \leq n \leq N-1 \quad (5.2.26)$$

Real and odd sequences. If $x(n)$ is real and odd, that is,

$$x(n) = -x(N-n) \quad 0 \leq n \leq N-1$$

then (5.2.20) yields $X_R(k) = 0$. Hence

$$X(k) = -j \sum_{n=0}^{N-1} x(n) \sin \frac{2\pi kn}{N} \quad 0 \leq k \leq N-1 \quad (5.2.27)$$

which is purely imaginary and odd. Since $X_R(k) = 0$, the IDFT reduces to

$$x(n) = j \frac{1}{N} \sum_{k=0}^{N-1} X(k) \sin \frac{2\pi kn}{N} \quad 0 \leq n \leq N-1 \quad (5.2.28)$$

Purely imaginary sequences. In this case, $x(n) = jx_I(n)$. Consequently, (5.2.20) and (5.2.21) reduce to

$$X_R(k) = \sum_{n=0}^{N-1} x_I(n) \sin \frac{2\pi kn}{N} \quad (5.2.29)$$

$$X_I(k) = \sum_{n=0}^{N-1} x_I(n) \cos \frac{2\pi kn}{N} \quad (5.2.30)$$

We observe that $X_R(k)$ is odd and $X_I(k)$ is even.

If $x_I(n)$ is odd, then $X_I(k) = 0$ and hence $X(k)$ is purely real. On the other hand, if $x_I(n)$ is even, then $X_R(k) = 0$ and hence $X(k)$ is purely imaginary.

TABLE 5.1 SYMMETRY PROPERTIES OF THE DFT

N-Point Sequence $x(n)$, $0 \leq n \leq N-1$	<i>N-Point DFT</i>
$x(n)$	$X(k)$
$x^*(n)$	$X^*(N-k)$
$x^*(N-n)$	$X^*(k)$
$x_R(n)$	$X_{er}(k) = \frac{1}{2}[X(k) + X^*(N-k)]$
$jX_I(n)$	$X_{eo}(k) = \frac{1}{2}[X(k) - X^*(N-k)]$
$x_{er}(n) = \frac{1}{2}[x(n) + x^*(N-n)]$	$X_R(k)$
$x_{eo}(n) = \frac{1}{2}[x(n) - x^*(N-n)]$	$jX_I(k)$
Any real signal $x(n)$	Real Signals $X(k) = X^*(N-k)$ $X_R(k) = X_R(N-k)$ $X_I(k) = -X_I(N-k)$ $ X(k) = X(N-k) $ $\angle X(k) = -\angle X(N-k)$
$x_{er}(n) = \frac{1}{2}[x(n) + x(N-n)]$	$X_R(k)$
$x_{eo}(n) = \frac{1}{2}[x(n) - x(N-n)]$	$jX_I(k)$

The symmetry properties given above may be summarized as follows:

$$\begin{aligned}
 x(n) &= x_R^e(n) + x_R^o(n) + jx_I^e(n) + jx_I^o(n) \\
 X(k) &= X_R^e(k) + X_R^o(k) + jX_I^e(k) + jX_I^o(k)
 \end{aligned}
 \tag{5.2.31}$$

All the symmetry properties of the DFT can easily be deduced from (5.2.31). For example, the **DFT** of the sequence

$$x_{pe}(n) = \frac{1}{2}[x_p(n) + x_p^*(N-n)]$$

$$X_R(k) = X_R^e(k) + X_R^o(k)$$

The symmetry properties of the DFT are summarized in Table 5.1. Exploitation of these properties for the efficient computation of the DFT of special sequences is considered in some of the problems at the end of the chapter.

5.2.2 Multiplication of Two DFTs and Circular Convolution

Suppose that we have two finite-duration sequences of length N , $x_1(n)$ and $x_2(n)$. Their respective N -point DFTs are

$$X_1(k) = \sum_{n=0}^{N-1} x_1(n)e^{-j2\pi nk/N} \quad k = 0, 1, \dots, N-1 \tag{5.2.32}$$

$$X_2(k) = \sum_{n=0}^{N-1} x_2(n)e^{-j2\pi nk/N} \quad k = 0, 1, \dots, N-1 \tag{5.2.33}$$

If we multiply the two DFTs together, the result is a DFT, say $X_3(k)$, of a sequence $x_3(n)$ of length N . Let us determine the relationship between $x_3(n)$ and the sequences $x_1(n)$ and $x_2(n)$.

We have

$$X_3(k) = X_1(k)X_2(k) \quad k = 0, 1, \dots, N-1 \quad (5.2.34)$$

The IDFT of $\{X_3(k)\}$ is

$$\begin{aligned} x_3(m) &= \frac{1}{N} \sum_{k=0}^{N-1} X_3(k) e^{j2\pi km/N} \\ &= \frac{1}{N} \sum_{k=0}^{N-1} X_1(k) X_2(k) e^{j2\pi km/N} \end{aligned} \quad (5.2.35)$$

Suppose that we substitute for $X_1(k)$ and $X_2(k)$ in (5.2.35) using the DFTs given in (5.2.32) and (5.2.33). Thus we obtain

$$\begin{aligned} x_3(m) &= \frac{1}{N} \sum_{k=0}^{N-1} \left[\sum_{n=0}^{N-1} x_1(n) e^{-j2\pi kn/N} \right] \left[\sum_{l=0}^{N-1} x_2(l) e^{-j2\pi kl/N} \right] e^{j2\pi km/N} \\ &= \frac{1}{N} \sum_{n=0}^{N-1} x_1(n) \sum_{l=0}^{N-1} x_2(l) \left[\sum_{k=0}^{N-1} e^{j2\pi k(m-n-l)/N} \right] \end{aligned} \quad (5.2.36)$$

The inner sum in the brackets in (5.2.36) has the form

$$\sum_{k=0}^{N-1} a^k = \begin{cases} N, & a = 1 \\ \frac{1-a^N}{1-a}, & a \neq 1 \end{cases} \quad (5.2.37)$$

where a is defined as

$$a = e^{j2\pi(m-n-l)/N}$$

We observe that $a = 1$ when $m - n - l$ is a multiple of N . On the other hand $a^N = 1$ for any value of $a \neq 0$. Consequently, (5.2.37) reduces to

$$\sum_{k=0}^{N-1} a^k = \begin{cases} N, & l = m - n + pN = ((m-n))_N, \text{ p an integer} \\ 0, & \text{otherwise} \end{cases} \quad (5.2.38)$$

If we substitute the result in (5.2.38) into (5.2.36), we obtain the desired expression for $x_3(m)$ in the form

$$x_3(m) = \sum_{n=0}^{N-1} x_1(n) x_2((m-n))_N \quad m = 0, 1, \dots, N-1 \quad (5.2.39)$$

The expression in (5.2.39) has the form of a convolution sum. However, it is not the ordinary linear convolution that was introduced in Chapter 2, which relates the output sequence $y(n)$ of a linear system to the input sequence $x(n)$ and the impulse response $h(n)$. Instead, the convolution sum in (5.2.39) involves the index

$((m-n))_N$ and is called **circular convolution**. Thus we conclude that multiplication of the DFTs of two sequences is equivalent to the circular convolution of the two sequences in the time domain.

The following example illustrates the operations involved in circular convolution.

Example 5.2.1

Perform the circular convolution of the following two sequences:

$$\begin{array}{c} x_1(n) = \{2, 1, 2, 1\} \\ \uparrow \\ x_2(n) = \{1, 2, 3, 4\} \\ \uparrow \end{array}$$

Solution Each sequence consists of four nonzero points. For the purposes of illustrating the operations involved in circular convolution, it is desirable to graph each sequence as points on a circle. Thus the sequences $x_1(n)$ and $x_2(n)$ are graphed as illustrated in Fig. 5.8(a). We note that the sequences are graphed in a counterclockwise direction on a circle. This establishes the reference direction in rotating one of the sequences relative to the other.

Now, $x_3(m)$ is obtained by circularly convolving $x_1(n)$ with $x_2(n)$ as specified by (5.2.39). Beginning with $m = 0$ we have

$$x_3(0) = \sum_{n=0}^3 x_1(n)x_2((-n))_4$$

$x_2((-n))_4$ is simply the sequence $x_2(n)$ folded and graphed on a circle as illustrated in Fig. 5.8(b). In other words, the folded sequence is simply $x_2(n)$ graphed in a clockwise direction.

The product sequence is obtained by multiplying $x_1(n)$ with $x_2((-n))_4$, point by point. This sequence is also illustrated in Fig. 5.8(b). Finally, we sum the values in the product sequence to obtain

$$x_3(0) = 14$$

For $m = 1$ we have

$$x_3(1) = \sum_{n=0}^3 x_1(n)x_2((1-n))_4$$

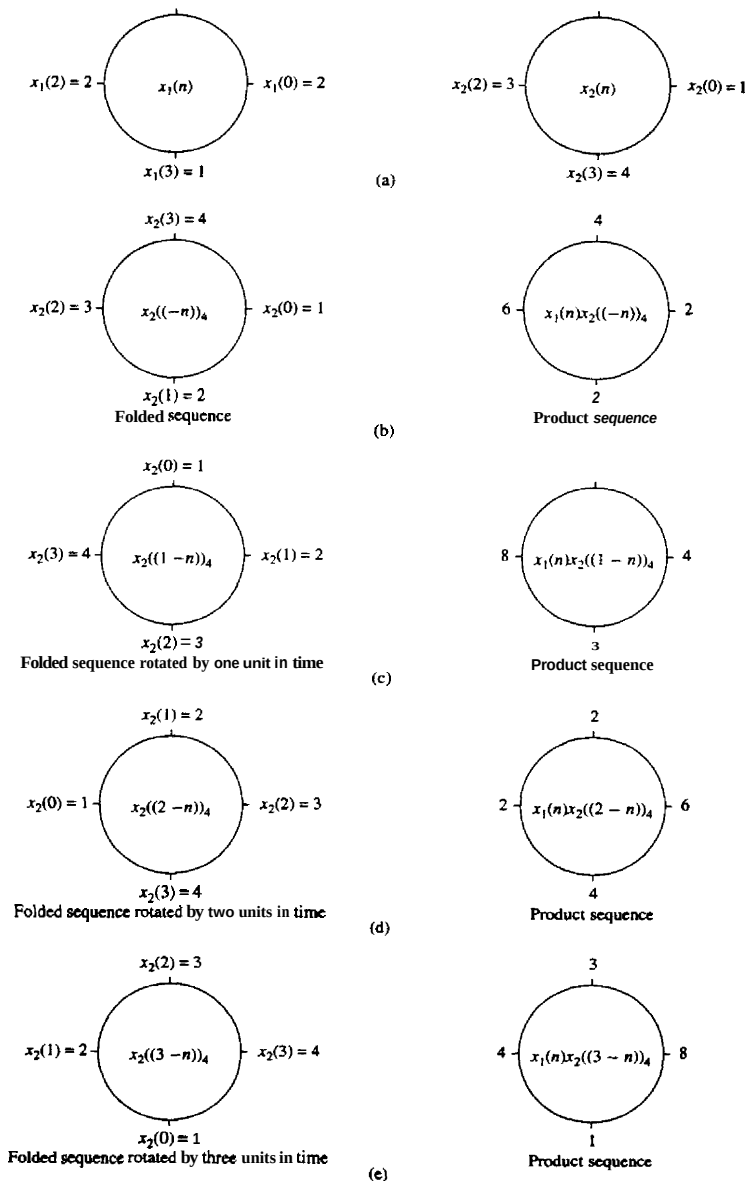
It is easily verified that $x_2((1-n))_4$ is simply the sequence $x_2((-n))_4$ rotated counterclockwise by one unit in time as illustrated in Fig. 5.8(c). This rotated sequence multiplies $x_1(n)$ to yield the product sequence, also illustrated in Fig. 5.8(c). Finally, we sum the values in the product sequence to obtain $x_3(1)$. Thus

$$x_3(1) = 16$$

For $m = 2$ we have

$$x_3(2) = \sum_{n=0}^3 x_1(n)x_2((2-n))_4$$

Now $x_2((2-n))_4$ is the folded sequence in Fig. 5.8(b) rotated two units of time in the counterclockwise direction. The resultant sequence is illustrated in Fig. 5.8(d)



along with the product sequence $x_1(n)x_2((2-n))_4$. By summing the four terms in the product sequence, we obtain

$$x_3(2) = 14$$

For $m = 3$ we have

$$x_3(3) = \sum_{n=0}^3 x_1(n)x_2((3-n))_4$$

The folded sequence $x_2((-n))_4$ is now rotated by three units in time to yield $x_2((3-n))_4$ and the resultant sequence is multiplied by $x_1(n)$ to yield the product sequence as illustrated in Fig. 5.8(e). The sum of the values in the product sequence is

$$x_3(3) = 16$$

We observe that if the computation above is continued beyond $m = 3$, we simply repeat the sequence of four values obtained above. Therefore, the circular convolution of the two sequences $x_1(n)$ and $x_2(n)$ yields the sequence

$$x_3(n) = \{14, 16, 14, 16\}$$

↑

From this example, we observe that circular convolution involves basically the same four steps as the ordinary **linear convolution** introduced in Chapter 2: **folding** (time reversing) one sequence, **shifting** the folded sequence, **multiplying** the two sequences to obtain a product sequence, and finally, **summing** the values of the product sequence. The basic difference between these two types of convolution is that, in circular convolution, the folding and shifting (rotating) operations are performed in a circular fashion by computing the index of one of the sequences modulo N . In linear convolution, there is no modulo N operation.

The reader can **easily** show from our previous development that either one of the two sequences may be folded and rotated without changing the result of the circular convolution. Thus

$$x_3(m) = \sum_{n=0}^{N-1} x_2(n)x_1((m-n))_N \quad m = 0, 1, \dots, N-1 \quad (5.2.40)$$

The following example serves to illustrate the computation of $x_3(n)$ by means of the **DFT** and **IDFT**.

Example 5.2.2

By means of the **DFT** and **IDFT**, **determine** the sequence $x_3(n)$ corresponding to the circular convolution of the sequences $x_1(n)$ and $x_2(n)$ given in Example 5.2.1.

Solution First we compute the **DFTs** of $x_1(n)$ and $x_2(n)$. The four-point **DFT** of $x_1(n)$ is

$$\begin{aligned} X_1(k) &= \sum_{n=0}^3 x_1(n)e^{-j2\pi nk/4} \quad k = 0, 1, 2, 3 \\ &= 2 + e^{-j\pi k/2} + 2e^{-j\pi k} + e^{-j3\pi k/2} \end{aligned}$$

Thus

$$X_1(0) = 6 \quad X_1(1) = 0 \quad X_1(2) = 2 \quad X_1(3) = 0$$

The DFT of $x_2(n)$ is

$$\begin{aligned} X_2(k) &= \sum_{n=0}^3 x_2(n) e^{-j2\pi nk/4} \quad k = 0, 1, 2, 3 \\ &= 1 + 2e^{-j\pi k/2} + 3e^{-j\pi k} + 4e^{-j3\pi k/2} \end{aligned}$$

Thus

$$X_2(0) = 10 \quad X_2(1) = -2 + j2 \quad X_2(2) = -2 \quad X_2(3) = -2 - j2$$

When we multiply the two DFTs, we obtain the product

$$X_3(k) = X_1(k)X_2(k)$$

or, equivalently,

$$X_3(0) = 60 \quad X_3(1) = 0 \quad X_3(2) = -4 \quad X_3(3) = 0$$

Now, the **IDFT** of $X_3(k)$ is

$$\begin{aligned} x_3(n) &= \sum_{k=0}^3 X_3(k) e^{j2\pi nk/4} \quad n = 0, 1, 2, 3 \\ &= \frac{1}{4}(60 - 4e^{j\pi n}) \end{aligned}$$

Thus

$$x_3(0) = 14 \quad x_3(1) = 16 \quad x_3(2) = 14 \quad x_3(3) = 16$$

which is the result obtained in Example 5.2.1 from circular *convolution*.

We conclude this section by formally stating this important property of the DFT.

Circular convolution. If

$$x_1(n) \xleftrightarrow[N]{\text{DFT}} X_1(k)$$

and

$$x_2(n) \xleftrightarrow[N]{\text{DFT}} X_2(k)$$

then

$$x_1(n) \circledast x_2(n) \xleftrightarrow[N]{\text{DFT}} X_1(k)X_2(k) \quad (5.2.41)$$

where $x_1(n) \circledast x_2(n)$ denotes the circular convolution of the sequence $x_1(n)$ and $x_2(n)$.

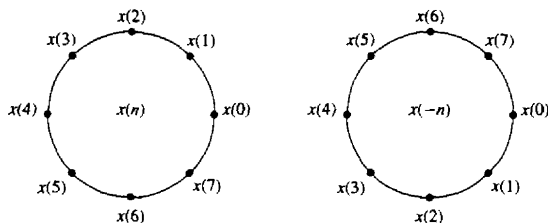


Figure 5.9 Time reversal of a sequence.

5.2.3 Additional DFT Properties

Time reversal of a sequence. If

$$x(n) \xleftrightarrow[N]{\text{DFT}} X(k)$$

then

$$x((-n))_N = x(N - n) \xleftrightarrow{\text{DFT}} X((-k))_N = X(N - k) \quad (5.2.42)$$

Hence reversing the N -point sequence in time is equivalent to reversing the DFT values. Time reversal of a sequence $x(n)$ is illustrated in Fig. 5.9.

Proof: From the definition of the DFT in (5.2.1) we have

$$\text{DFT}\{x(N - n)\} = \sum_{n=0}^{N-1} x(N - n) e^{-j2\pi kn/N}$$

If we change the index from n to $m = N - n$, then

$$\begin{aligned} \text{DFT}\{x(N - n)\} &= \sum_{m=0}^{N-1} x(m) e^{-j2\pi k(N-m)/N} \\ &= \sum_{m=0}^{N-1} x(m) e^{j2\pi km/N} \\ &= \sum_{m=0}^{N-1} x(m) e^{-j2\pi m(N-k)/N} = X(N - k) \end{aligned}$$

We note that $X(N - k) = X((-k))_N$, $0 \leq k \leq N - 1$.

Circular time shift of a sequence. If

$$x(n) \xleftrightarrow[N]{\text{DFT}} X(k)$$

then

$$x((n-l))_N \xleftrightarrow{N} X(k)e^{-j2\pi kl/N} \quad (5.2.43)$$

Proof. From the definition of the DFT we have

$$\begin{aligned} \text{DFT}\{x((n-l))_N\} &= \sum_{n=0}^{N-1} x((n-l))_N e^{-j2\pi kn/N} \\ &= \sum_{n=0}^{l-1} x((n-l))_N e^{-j2\pi kn/N} \\ &\quad + \sum_{n=l}^{N-1} x(n-l) e^{-j2\pi kn/N} \end{aligned}$$

But $x((n-l))_N = x(N-l+n)$. Consequently,

$$\begin{aligned} \sum_{n=0}^{l-1} x((n-l))_N e^{-j2\pi kn/N} &= \sum_{n=l}^{N-1} x(N-l+n) e^{-j2\pi kn/N} \\ &= \sum_{m=N-l}^{N-1} x(m) e^{-j2\pi k(m+l)/N} \end{aligned}$$

Furthermore,

$$\sum_{n=l}^{N-1} x(n-l) e^{-j2\pi kn/N} = \sum_{m=0}^{N-1-l} x(m) e^{-j2\pi k(m+l)/N}$$

Therefore,

$$\begin{aligned} \text{DFT}\{x((n-l))\} &= \sum_{m=0}^{N-1} x(m) e^{-j2\pi k(m+l)/N} \\ &= X(k) e^{-j2\pi kl/N} \end{aligned}$$

Circular frequency shift. If

$$x(n) \xleftrightarrow{N} X(k)$$

then

$$x(n) e^{j2\pi ln/N} \xleftrightarrow{N} X((k-l))_N \quad (5.2.44)$$

Hence, the multiplication of the sequence $x(n)$ with the complex exponential sequence $e^{j2\pi ln/N}$ is equivalent to the circular shift of the DFT by l units in frequency. This is the dual to the circular time-shifting property and its proof is similar to the latter.

Complex-conjugate properties. If

$$x(n) \xleftrightarrow[N]{\text{DFT}} X(k)$$

then

$$x^*(n) \xleftrightarrow[N]{\text{DFT}} X^*((-k))_N = X^*(N-k) \quad (5.2.45)$$

The proof of this property is left as an exercise for the reader. The IDFT of $X^*(k)$ is

$$\frac{1}{N} \sum_{k=0}^{N-1} X^*(k) e^{j2\pi kn/N} = \left[\frac{1}{N} \sum_{k=0}^{N-1} X(k) e^{j2\pi k(N-n)/N} \right]^*$$

Therefore,

$$x^*((-n))_N = x^*(N-n) \xleftrightarrow{\text{DFT}} X^*(k) \quad (5.2.46)$$

Circular correlation. In general, for complex-valued sequences $x(n)$ and $y(n)$, if

$$x(n) \xleftrightarrow{\text{DFT}} X(k)$$

and

$$y(n) \xleftrightarrow[N]{\text{DFT}} Y(k)$$

then

$$\tilde{r}_{xy}(l) \xleftrightarrow[N]{\text{DFT}} \tilde{R}_{xy}(k) = X(k)Y^*(k) \quad (5.2.47)$$

where $\tilde{r}_{xy}(l)$ is the (unnormalized) circular crosscorrelation sequence, defined as

$$\tilde{r}_{xy}(l) = \sum_{n=0}^{N-1} x(n)y^*((n-l))_N$$

Proof: We can write $\tilde{r}_{xy}(l)$ as the circular convolution of $x(n)$ with $y^*(-n)$, that is,

$$\tilde{r}_{xy}(l) = x(l) \circledast y^*(-l)$$

Then, with the aid of the properties in (5.2.41) and (5.2.46), the N-point DFT of $\tilde{r}_{xy}(l)$ is

$$\tilde{R}_{xy}(k) = X(k)Y^*(k)$$

In the special case where $y(n) = x(n)$, we have the corresponding expression for the circular autocorrelation of $x(n)$,

$$\tilde{r}_{xx}(l) \xleftrightarrow[N]{\text{DFT}} \tilde{R}_{xx}(k) = |X(k)|^2 \quad (5.2.48)$$

Multiplication of two sequences. If

$$x_1(n) \xleftrightarrow[N]{\text{DFT}} X_1(k)$$

and

$$x_2(n) \xleftrightarrow[N]{\text{DFT}} X_2(k)$$

then

$$x_1(n)x_2(n) \xleftrightarrow[N]{\text{DFT}} \frac{1}{N} X_1(k) \circledast X_2(k) \quad (5.2.49)$$

This property is the dual of (5.2.41). Its proof follows simply by interchanging the roles of time and frequency in the expression for the circular convolution of two sequences.

Parseval's theorem. For complex-valued sequences $x(n)$ and $y(n)$, in general, if

$$x(n) \xleftrightarrow[N]{\text{DFT}} X(k)$$

and

$$y(n) \xleftrightarrow[N]{\text{DFT}} Y(k)$$

then

$$\sum_{n=0}^{N-1} x(n)y^*(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k)Y^*(k) \quad (5.2.50)$$

Proof The property follows immediately from the circular correlation property in (5.2.47). We have

$$\sum_{n=0}^{N-1} x(n)y^*(n) = \tilde{r}_{xy}(0)$$

and

$$\begin{aligned} \tilde{r}_{xy}(l) &= \frac{1}{N} \sum_{k=0}^{N-1} \tilde{R}_{xy}(k) e^{j2\pi kl/N} \\ &= \frac{1}{N} \sum_{k=0}^{N-1} X(k)Y^*(k) e^{j2\pi kl/N} \end{aligned}$$

Hence (5.2.50) follows by **evaluating** the IDFT at $l = 0$.

The expression in (5.2.50) is the general form of Parseval's theorem. In the special case where $y(n) = x(n)$, (5.2.50) reduces to

$$\sum_{n=0}^{N-1} |x(n)|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |X(k)|^2 \quad (5.2.51)$$

TABLE 5.2 PROPERTIES OF THE DFT

Property	Time Domain	Frequency Domain
Notation	$x(n), y(n)$	$X(k), Y(k)$
Periodicity	$x(n) = x(n + N)$	$X(k) = X(k + N)$
Linearity	$a_1 x_1(n) + a_2 x_2(n)$	$a_1 X_1(k) + a_2 X_2(k)$
Time reversal	$x(N - n)$	$X(N - k)$
Circular time shift	$x((n - l))_N$	$X(k) e^{-j2\pi kl/N}$
Circular frequency shift	$x(n) e^{j2\pi ln/N}$	$X((k - l))_N$
Complex conjugate	$x^*(n)$	$X^*(N - k)$
Circular convolution	$x_1(n) \circledast x_2(n)$	$X_1(k) X_2(k)$
Circular correlation	$x(n) \circledast y^*(-n)$	$X(k) Y^*(k)$
Multiplication of two sequences	$x_1(n) x_2(n)$	$\frac{1}{N} X_1(k) \circledast X_2(k)$
Parseval's theorem	$\sum_{n=0}^{N-1} x(n) y^*(n)$	$\frac{1}{N} \sum_{k=0}^{N-1} X(k) Y^*(k)$

which expresses the energy in the finite-duration sequence $x(n)$ in terms of the frequency components $\{X(k)\}$.

The properties of the DFT given above are summarized in Table 5.2.

5.3 LINEAR FILTERING METHODS BASED ON THE DFT

Since the DFT provides a discrete frequency representation of a finite-duration sequence in the frequency domain, it is interesting to explore its use as a computational tool for linear system analysis and, especially, for linear filtering. We have already established that a system with frequency response $H(\omega)$, when excited with an input signal that has a spectrum $X(\omega)$, possesses an output spectrum $Y(\omega) = X(\omega)H(\omega)$. The output sequence $y(n)$ is determined from its spectrum via the inverse Fourier transform. **Computationally**, the problem with this **frequency-domain** approach is that $X(\omega)$, $H(\omega)$, and $Y(\omega)$ are functions of the continuous variable ω . As a consequence, the computations cannot be done on a digital **computer**, since the computer can only store and perform computations on quantities at discrete frequencies.

On the other hand, the DFT does **lend** itself to computation on a digital computer. In the discussion that follows, we describe how the DFT **can** be used to perform linear filtering in the frequency domain. In particular, we present a computational procedure that serves **as** an alternative to time-domain convolution. In fact, the **frequency-domain** approach based on the DFT, is **computationally more** efficient than time-domain convolution due to the existence of efficient algorithms for computing the **DFT**. These algorithms, which are **described** in Chapter 6, are collectively called fast Fourier **transform** (FFT) algorithms,

5.3.1 Use of the DFT in Linear Filtering

In the preceding section it was demonstrated that the product of two DFTs is equivalent to the circular convolution of the corresponding time-domain sequences. Unfortunately, circular convolution is of no use to us if our objective is to determine the output of a linear filter to a given input sequence. In this case we seek a frequency-domain methodology equivalent to linear convolution.

Suppose that we have a finite-duration sequence $x(n)$ of length L which excites an **FIR** filter of length M . Without loss of generality, let

$$x(n) = 0, \quad n < 0 \text{ and } n \geq L$$

$$h(n) = 0, \quad n < 0 \text{ and } n \geq M$$

where $h(n)$ is the impulse response of the FIR filter.

The output sequence $y(n)$ of the FIR filter can be expressed in the time domain as the convolution of $x(n)$ and $h(n)$, that is

$$y(n) = \sum_{k=0}^{M-1} h(k)x(n-k) \quad (5.3.1)$$

Since $h(n)$ and $x(n)$ are finite-duration sequences, their convolution is also finite in duration. In fact, the duration of $y(n)$ is $L + M - 1$.

The frequency-domain equivalent to (5.3.1) is

$$Y(\omega) = X(\omega)H(\omega) \quad (5.3.2)$$

If the sequence $y(n)$ is to be represented uniquely in the frequency domain by samples of its spectrum $Y(\omega)$ at a set of discrete frequencies, the number of distinct samples must equal or exceed $L + M - 1$. Therefore, a **DFT** of size $N \geq L + M - 1$, is required to represent $\{y(n)\}$ in the frequency domain.

Now if

$$\begin{aligned} Y(k) &\equiv Y(\omega)|_{\omega=2\pi k/N} & k &= 0, 1, \dots, N-1 \\ &= X(\omega)H(\omega)|_{\omega=2\pi k/N} & k &= 0, 1, \dots, N-1 \end{aligned}$$

then

$$Y(k) = X(k)H(k) \quad k = 0, 1, \dots, N-1 \quad (5.3.3)$$

where $\{X(k)\}$ and $\{H(k)\}$ are the N -point DFTs of the corresponding sequences $x(n)$ and $h(n)$, respectively. Since the sequences $x(n)$ and $h(n)$ have a duration less than N , we **simply** pad these sequences with zeros to increase their length to N . This increase in the size of the sequences does not alter their spectra $X(\omega)$ and $H(\omega)$, which are continuous spectra, since the sequences are aperiodic. However, by sampling their spectra at N equally spaced points in frequency (computing the N -point DFTs), we have increased the number of samples that represent **these** sequences in the frequency domain beyond the minimum number (L or M , respectively).

Since the $N = L + M - 1$ -point DFT of the output sequence $y(n)$ is sufficient to represent $y(n)$ in the frequency domain, it follows that the multiplication of the N -point DFTs $X(k)$ and $H(k)$, according to (5.3.3), followed by the computation of the N -point IDFT, must yield the sequence $\{y(n)\}$. In turn, this implies that the N -point circular convolution of $x(n)$ with $h(n)$ must be equivalent to the linear convolution of $x(n)$ with $h(n)$. In other words, by increasing the length of the sequences $x(n)$ and $h(n)$ to N points (by appending zeros), and then circularly convolving the resulting sequences, we obtain the same result as would have been obtained with linear convolution. Thus with zero padding, the DFT can be used to perform linear filtering.

The following example illustrates the methodology in the use of the DFT in linear filtering.

Example 5.3.1

By means of the DFT and IDFT, determine the response of the FIR filter with impulse response

$$h(n) = \{1, 2, 3\}$$

↑

to the input sequence

$$x(n) = \{1, 2, 2, 1\}$$

↑

Solution The input sequence has length $L = 4$ and the impulse response has length $M = 3$. Linear convolution of these two sequences produces a sequence of length $N = 6$. Consequently, the size of the DFTs must be at least six.

For simplicity we compute eight-point DFTs. We should also mention that the efficient computation of the DFT via the fast Fourier transform (FFT) algorithm is usually performed for a length N that is a power of 2. Hence the eight-point DFT of $x(n)$ is

$$\begin{aligned} X(k) &= \sum_{n=0}^7 x(n)e^{-j2\pi kn/8} \\ &= 1 + 2e^{-j\pi k/4} + 2e^{-j\pi k/2} + e^{-j3\pi k/4} \quad k = 0, 1, \dots, 7 \end{aligned}$$

This computation yields

$$\begin{aligned} X(0) &= 6 & X(1) &= \frac{2 + \sqrt{2}}{2} - j \left(\frac{4 + 3\sqrt{2}}{2} \right) \\ X(2) &= -1 - j & X(3) &= \frac{2 - \sqrt{2}}{2} + j \left(\frac{4 - 3\sqrt{2}}{2} \right) \\ X(4) &= 0 & X(5) &= \frac{2 - \sqrt{2}}{2} - j \frac{4 - 3\sqrt{2}}{2} \\ X(6) &= -1 + j & X(7) &= \frac{2 + \sqrt{2}}{2} + j \left(\frac{4 + 3\sqrt{2}}{2} \right) \end{aligned}$$

The eight-point DFT of $h(n)$ is

$$H(k) = \sum_{n=0}^7 h(n)e^{-j2\pi kn/8} \\ = 1 + 2e^{-j\pi k/4} + 3e^{-j\pi k/2}$$

Hence

$$H(0) = 6, \quad H(1) = 1 + \sqrt{2} - j(3 + \sqrt{2}), \quad H(2) = -2 - j2$$

$$H(3) = 1 - \sqrt{2} + j(3 - \sqrt{2}), \quad H(4) = 2$$

$$H(5) = 1 - \sqrt{2} - j(3 - \sqrt{2}), \quad H(6) = -2 + j2$$

$$H(7) = 1 + \sqrt{2} + j(3 + \sqrt{2})$$

The product of these two DFTs yields $Y(k)$, which is

$$Y(0) = 36, \quad Y(1) = -14.07 - j17.48 \quad Y(2) = j4 \quad Y(3) = 0.07 + j0.515$$

$$Y(4) = 0, \quad Y(5) = 0.07 - j0.515 \quad Y(6) = -j4 \quad Y(7) = -14.07 + j17.48$$

Finally, the eight-point IDFT is

$$y(n) = \sum_{k=0}^7 Y(k)e^{j2\pi kn/8} \quad n = 0, 1, \dots, 7$$

This computation yields the result

$$y(n) = \{1, 4, 9, 11, 8, 3, 0, 0\}$$

We observe that the first six values of $y(n)$ constitute the set of desired output values. The last two values are zero because we used an eight-point DFT and IDFT, when, in fact, the minimum number of points required is six.

Although the multiplication of two DFTs corresponds to circular convolution in the time domain, we have observed that padding the sequences $x(n)$ and $h(n)$ with a sufficient number of zeros forces the circular convolution to yield the same output sequence as linear convolution. In the case of the FIR filtering problem in Example 5.3.1, it is a simple matter to demonstrate that the six-point circular convolution of the sequences

$$h(n) = \{1, 2, 3, 0, 0, 0\} \quad (5.3.4)$$

$$x(n) = \{1, 2, 2, 1, 0, 0\} \quad (5.3.5)$$

results in the output sequence

$$y(n) = \{1, 4, 9, 11, 8, 3\} \quad (5.3.6)$$

which is the same sequence obtained from linear convolution.

It is important for us to understand the aliasing that results in the time domain when the size of the DFTs is smaller than $L + M - 1$. The following example focuses on the aliasing problem.

Example 5.3.2

Determine the sequence $y(n)$ that results from the use of four point **DFTs** in Example 5.3.1.

Solution The four-point DFT of $h(n)$ is

$$H(k) = \sum_{n=0}^3 h(n)e^{-j2\pi kn/4}$$

$$H(k) = 1 + 2e^{-j\pi k/2} + 3e^{-jk\pi} \quad k = 0, 1, 2, 3$$

Hence

$$H(0) = 6, \quad H(1) = -2 - j2, \quad H(2) = 2, \quad H(3) = -2 + j2$$

The four-point **DFT** of $x(n)$ is

$$X(k) = 1 + 2e^{-j\pi k/2} + 2e^{-jk\pi} + 3e^{-j3\pi k/2} \quad k = 0, 1, 2, 3$$

Hence

$$X(0) = 6, \quad X(1) = -1 - j, \quad X(2) = 0, \quad X(3) = -1 + j$$

The product of these two four-point **DFTs** is

$$\hat{Y}(0) = 36, \quad \hat{Y}(1) = j4, \quad \hat{Y}(2) = 0, \quad \hat{Y}(3) = -j4$$

The four-point **IDFT** yields

$$\hat{y}(n) = \frac{1}{4} \sum_{k=0}^3 \hat{Y}(k)e^{j2\pi kn/4} \quad n = 0, 1, 2, 3$$

$$= \frac{1}{4}(36 + j4e^{j\pi n/2} - j4e^{j3\pi n/2})$$

Therefore,

$$\hat{y}(n) = \{9, 7, 9, 11\}$$

$$\uparrow$$

The reader can verify that the four-point circular convolution of $h(n)$ with $x(n)$ yields the same sequence $\hat{y}(n)$.

If we compare the result $\hat{y}(n)$, obtained from four-point DFTs with the sequence $y(n)$ obtained from the use of eight-point (or six-point) DFTs, the time-domain aliasing effects derived in Section 5.2.2 are clearly evident. In particular, $y(4)$ is aliased into $y(0)$ to yield

$$\hat{y}(0) = y(0) + y(4) = 9$$

Similarly, $y(5)$ is **aliased** into $y(1)$ to yield

$$\hat{y}(1) = y(1) + y(5) = 7$$

All other aliasing has no effect since $y(n) = 0$ for $n \geq 6$. Consequently, we have

$$\hat{y}(2) = y(2) = 9$$

$$\hat{y}(3) = y(3) = 11$$

Therefore, **only** the first two points of $\hat{y}(n)$ are corrupted **by** the effect of aliasing [i.e., $\hat{y}(0) \neq y(0)$ and $\hat{y}(1) \neq y(1)$]. This observation has important ramifications in the discussion of the following section, in which we treat the filtering of long sequences.

5.3.2 Filtering of Long Data Sequences

In practical applications involving linear filtering of signals, the input sequence $x(n)$ is often a very long sequence. This is especially true in some real-time signal processing applications concerned with signal monitoring and analysis.

Since linear filtering performed via the DFT involves operations on a block of data, which by necessity must be limited in size due to limited memory of a digital computer, a long input signal sequence must be segmented to fixed-size blocks prior to processing. Since the filtering is linear, successive blocks can be processed one at a time via the DFT and the output blocks are fitted together to **form** the overall output signal sequence.

We now describe two methods for linear **FIR** filtering a long sequence on a block-by-block basis using the DFT. The input sequence is segmented into blocks and each block is processed via the DFT and IDFT to produce a block of output data. The output blocks are fitted together to form an overall output sequence which is identical to the sequence obtained if the long block had been processed via time-domain convolution.

The two methods are **called** the overlap-save method and the overlap-odd **method**. For both methods we assume that the **FIR** filter has duration M . The input data sequence is segmented into blocks of L points, where, by assumption, $L \gg M$ without loss of generality.

Overlap-save method. In this method the size of the input data blocks is $N = L + M - 1$ and the **size** of the **DFTs** and IDFT are of length N . Each data block consists of the last $M - 1$ data points of the previous data block followed by L new data points to form a data sequence of length $N = L + M - 1$. **An** N -point **DFT** is computed for each data block. The impulse response of the **FIR** filter is increased in length by appending $L - 1$ zeros and an N -point **DFT** of the sequence is computed once and stored. The multiplication of the two N -point **DFTs** $\{H(k)\}$ and $\{X_m(k)\}$ for the m th block of data yields

$$\hat{Y}_m(k) = H(k)X_m(k) \quad k = 0, 1, \dots, N - 1 \quad (5.3.7)$$

Then the N -point **IDFT** yields the result

$$\hat{Y}_m(n) = \{\hat{y}_m(0)\hat{y}_m(1) \cdots \hat{y}_m(M-1)\hat{y}_m(M) \cdots \hat{y}_m(N-1)\} \quad (5.3.8)$$

Since the data record is of length N , the first $M - 1$ points of $y_m(n)$ are corrupted by aliasing and must be discarded. The last L points of $y_m(n)$ are exactly the same as the result from linear convolution and, as a consequence,

$$\hat{y}_m(n) = y_m(n), n = M, M + 1, \dots, N - 1 \quad (5.3.9)$$

To avoid loss of data due to aliasing, the last $M - 1$ points of each data record are saved and these points become the first $M - 1$ data points of the subsequent record, as indicated above. To begin the processing, the first $M - 1$ points of the first record are set to zero. Thus the blocks of data sequences are

$$x_1(n) = \underbrace{\{0, 0, \dots, 0\}}_{M-1 \text{ points}}, x(0), x(1), \dots, x(L - 1) \quad (5.3.10)$$

$$x_2(n) = \underbrace{\{x(L - M + 1), \dots, x(L - 1)\}}_{M-1 \text{ data points from } x_1(n)}, \underbrace{\{x(L), \dots, x(2L - 1)\}}_{L \text{ new data points}} \quad (5.3.11)$$

$$x_3(n) = \underbrace{\{x(2L - M + 1), \dots, x(2L - 1)\}}_{M-1 \text{ data points from } x_2(n)}, \underbrace{\{x(2L), \dots, x(3L - 1)\}}_{L \text{ new data points}} \quad (5.3.12)$$

and so forth. The resulting data sequences from the IDFT are given by (5.3.8), where the first $M - 1$ points are discarded due to aliasing and the remaining L points constitute the desired result from linear convolution. This segmentation of the input data and the fitting of the output data blocks together to form the output sequence are graphically illustrated in Fig. 5.10.

Overlap-add method. In this method the size of the input data block is L points and the size of the DFTs and IDFT is $N = L + M - 1$. To each data block we append $M - 1$ zeros and compute the N -point DFT. Thus the data blocks may be represented as

$$x_1(n) = \{x(0), x(1), \dots, x(L - 1), \underbrace{0, 0, \dots, 0}_{M-1 \text{ zeros}}\} \quad (5.3.13)$$

$$x_2(n) = \{x(L), x(L + 1), \dots, x(2L - 1), \underbrace{0, 0, \dots, 0}_{M-1 \text{ zeros}}\} \quad (5.3.14)$$

$$x_3(n) = \{x(2L), \dots, x(3L - 1), \underbrace{0, 0, \dots, 0}_{M-1 \text{ zeros}}\} \quad (5.3.15)$$

and so on. The two N -point DFTs are multiplied together to form

$$Y_m(k) = H(k)X_m(k) \quad k = 0, 1, \dots, N - 1 \quad (5.3.16)$$

The IDFT yields data blocks of length N that are free of aliasing since the size of the DFTs and IDFT is $N = L + M - 1$ and the sequences are increased to N -points by appending zeros to each block.

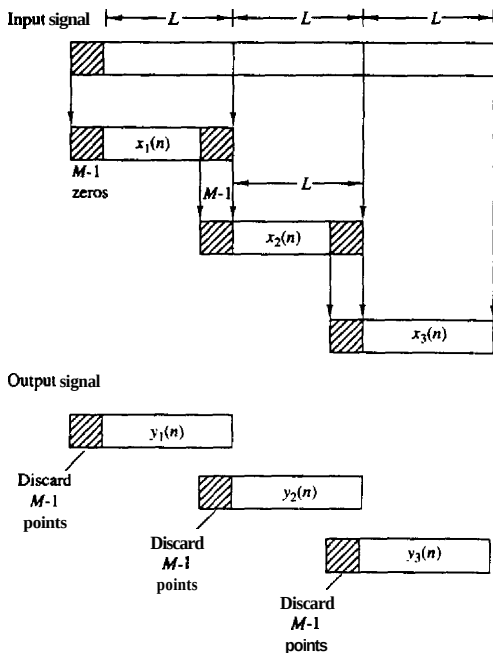


figure 5.10 Linear FIR filtering by the overlap-save method.

Since each data block is terminated with $M - 1$ zeros, the **last** $M - 1$ points from each output block must be overlapped and added to the first $M - 1$ points of the succeeding block. Hence this method is called the overlap-add method. This overlapping and adding yields the output sequence

$$y(n) = \{y_1(0), y_1(1), \dots, y_1(L - 1), y_1(L) + y_2(0), y_1(L + 1) + y_2(1), \dots, y_1(N - 1) + y_2(M - 1), y_2(M), \dots\} \quad (5.3.17)$$

The segmentation of the input data into blocks and the fitting of the output data blocks to **form** the output sequence are graphically illustrated in Fig. 5.11.

At this point, it may appear to the reader that the use of the DFT in linear **FIR** filtering is not only an indirect method of computing the output of an **FIR** filter, but it may also be more expensive computationally since the input data must first be converted to the frequency domain via the DFT, multiplied by **the DFT** of the FIR filter, and **finally**, converted back to the time domain via the IDFT. On the contrary, however, by **using** the fast Fourier transform algorithm, as **will** be shown in Chapter 6, the DFTs and IDFT require fewer computations to compute the output sequence than the direct realization of the **FIR** filter in the **time**

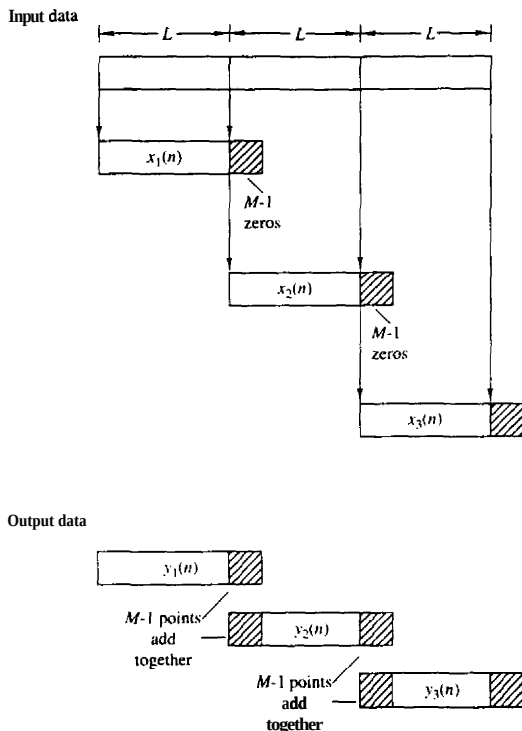


Figure 5.11 Linear FIR filtering by the overlap-add method.

domain. This computational efficiency is the basic advantage of using the DFT to compute the output of an FIR filter.

5.4 FREQUENCY ANALYSIS OF SIGNALS USING THE DFT

To compute the spectrum of either a continuous-time or discrete-time signal, the values of the signal for all time are required. However, in practice, we observe signals for only a finite duration. Consequently, the spectrum of a signal can only be approximated from a finite data record. In this section we examine the implications of a finite data record in frequency analysis using the DFT.

If the signal to be analyzed is an analog signal, we would first pass it through an **antialiasing** filter and then sample it at a rate $F_s \geq 2B$, where B is the bandwidth of the filtered signal. Thus the highest frequency that is contained in the sampled signal is $F_s/2$. Finally, for practical purposes, we **limit** the duration of the signal to the time interval $T_0 = LT$, where L is the number of samples and T

is the sample interval. As we shall observe in the following discussion, the finite observation interval for the signal **places** a limit on the frequency resolution; that is, it limits our ability to distinguish two frequency components that are separated by less than $1/T_0 = 1/LT$ in frequency.

Let $\{x(n)\}$ denote the sequence to be analyzed. Limiting the duration of the sequence to L samples, in the interval $0 \leq n \leq L-1$, is equivalent to multiplying $\{x(n)\}$ by a rectangular window $w(n)$ of length L . That is,

$$\hat{x}(n) = x(n)w(n) \quad (5.4.1)$$

where

$$w(n) = \begin{cases} 1, & 0 \leq n \leq L-1 \\ 0, & \text{otherwise} \end{cases} \quad (5.4.2)$$

Now suppose that the sequence $x(n)$ consists of a single sinusoid, that is,

$$x(n) = \cos \omega_0 n \quad (5.4.3)$$

Then the Fourier transform of the finite-duration sequence $x(n)$ can be expressed as

$$\hat{X}(\omega) = \frac{1}{2}[W(\omega - \omega_0) + W(\omega + \omega_0)] \quad (5.4.4)$$

where $W(\omega)$ is the Fourier transform of the window sequence, which is (for the rectangular window)

$$W(\omega) = \frac{\sin(\omega L/2)}{\sin(\omega/2)} e^{-j\omega(L-1)/2} \quad (5.4.5)$$

To compute $\hat{X}(\omega)$ we use the DFT. By padding the sequence $\hat{x}(n)$ with $N-L$ zeros, we can compute the N -point DFT of the truncated (L points) sequence $\{\hat{x}(n)\}$. The magnitude spectrum $|\hat{X}(k)| = |\hat{X}(\omega_k)|$ for $\omega_k = 2\pi k/N$, $k = 0, 1, \dots, N$, is illustrated in Fig. 5.12 for $L = 25$ and $N = 2048$. We note that the windowed spectrum $\hat{X}(\omega)$ is not localized to a single frequency, but instead it is spread out over the whole frequency range. Thus the power of the original signal sequence $\{x(n)\}$ that was concentrated at a single frequency has been spread by the window into the entire frequency range. We say that the power has "leaked out" into the entire frequency range. Consequently, this phenomenon, which is a characteristic of windowing the signal, is called leakage.

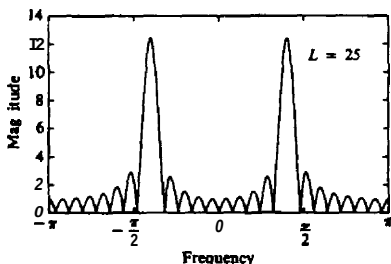


Figure 5.12 Magnitude spectrum for $L = 25$ and $n = 2048$, illustrating the occurrence of leakage.

Windowing not only distorts the spectral estimate due to the leakage effects, it also reduces spectral resolution. To illustrate this problem, let us consider a signal sequence consisting of two frequency components,

$$x(n) = \cos \omega_1 n + \cos \omega_2 n \quad (5.4.6)$$

When this sequence is truncated to L samples in the range $0 \leq n \leq L-1$, the windowed spectrum is

$$\hat{X}(\omega) = \frac{1}{2}[W(\omega - \omega_1) + W(\omega - \omega_2) + W(\omega + \omega_1) + W(\omega + \omega_2)] \quad (5.4.7)$$

The spectrum $W(\omega)$ of the rectangular window sequence has its first zero crossing at $\omega = 2\pi/L$. Now if $|\omega_1 - \omega_2| < 2\pi/L$, the two window functions $W(\omega - \omega_1)$ and $W(\omega - \omega_2)$ overlap and, as a consequence, the two spectral lines in $x(n)$ are not distinguishable. Only if $(\omega_1 - \omega_2) \geq 2\pi/L$ will we see two separate lobes in the spectrum $\hat{X}(\omega)$. Thus our ability to resolve spectral lines of different frequencies is limited by the window main lobe width. Figure 5.13 illustrates the magnitude spectrum $|\hat{X}(\omega)|$, computed via the DFT, for the sequence

$$x(n) = \cos \omega_0 n + \cos \omega_1 n + \cos \omega_2 n \quad (5.4.8)$$

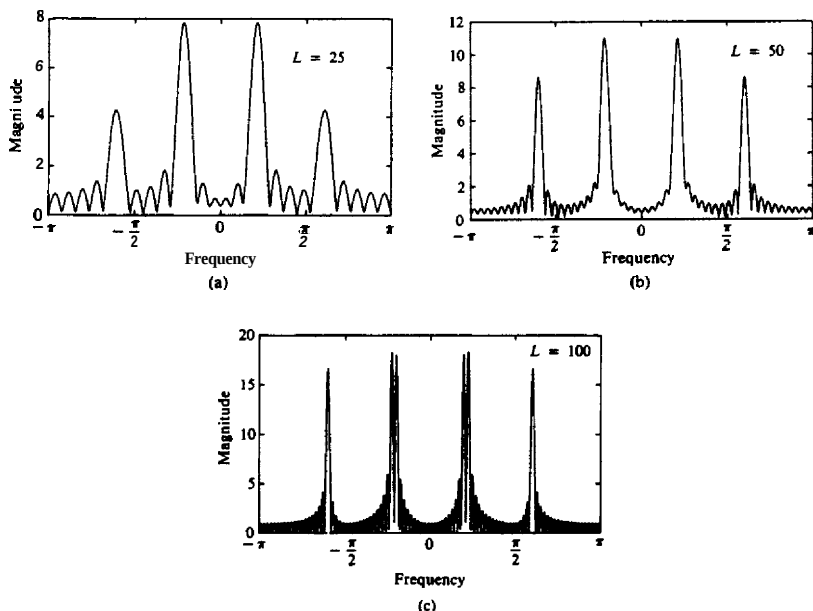


Figure 5.13 Magnitude spectrum for the signal given by (5.4.8), as observed through a rectangular window.

where $\omega_0 = 0.2\pi$, $\omega_1 = 0.22\pi$, and $\omega_2 = 0.6\pi$. The window lengths selected are $L = 25, 50$, and 100 . Note that ω_0 and ω_1 are not resolvable for $L = 25$ and 50 , but they are resolvable for $L = 100$.

To reduce leakage, we can select a data window $w(n)$ that has lower **sidelobes** in the frequency domain compared with the rectangular window. However, as we describe in more detail in Chapter 8, a reduction of the sidelobes in a window $W(\omega)$ is obtained at the expense of an increase in the width of the main lobe of $W(\omega)$ and hence a loss in resolution. To illustrate this point, let us consider the Hanning window, which is specified as

$$w(n) = \begin{cases} \frac{1}{2}(1 - \cos \frac{2\pi}{L-1}n), & 0 \leq n \leq L-1 \\ 0, & \text{otherwise} \end{cases} \quad (5.4.9)$$

Figure 5.14 shows $|\hat{X}(\omega)|$ for the window of (5.4.9). Its sidelobes are significantly smaller than those of the rectangular window, but its main lobe is approximately twice as wide. Figure 5.15 shows the spectrum of the signal in (5.4.8), after it is windowed by the Hanning window, for $L = 50, 75$, and 100 . The reduction of the sidelobes and the decrease in the resolution, compared with the rectangular window, is clearly evident.

For a general signal sequence $\{x(n)\}$, the frequency-domain relationship between the windowed sequence $\hat{x}(n)$ and the original sequence $x(n)$ is given by the convolution formula

$$\hat{X}(\omega) = \frac{1}{2\pi} \int_{-\pi}^{\pi} X(\theta) W(\omega - \theta) d\theta \quad (5.4.10)$$

The DFT of the windowed sequence $\hat{x}(n)$ is the sampled version of the spectrum $X(\omega)$. Thus we have

$$\begin{aligned} \hat{X}(k) &\equiv \hat{X}(\omega)|_{\omega=2\pi k/N} \\ &= \frac{1}{2\pi} \int_{-\pi}^{\pi} X(\theta) W\left(\frac{2\pi k}{N} - \theta\right) d\theta \quad k = 0, 1, \dots, N-1 \end{aligned} \quad (5.4.11)$$

Just as in the case of the sinusoidal sequence, if the spectrum of the window is relatively narrow in width compared to the spectrum $X(\omega)$ of the signal, the window function has only a small (smoothing) effect on the spectrum $X(\omega)$. On the other hand, if the window function has a wide spectrum compared to the width of

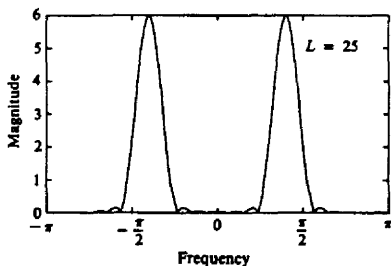


Figure 5.14 Magnitude spectrum of the Hanning window.

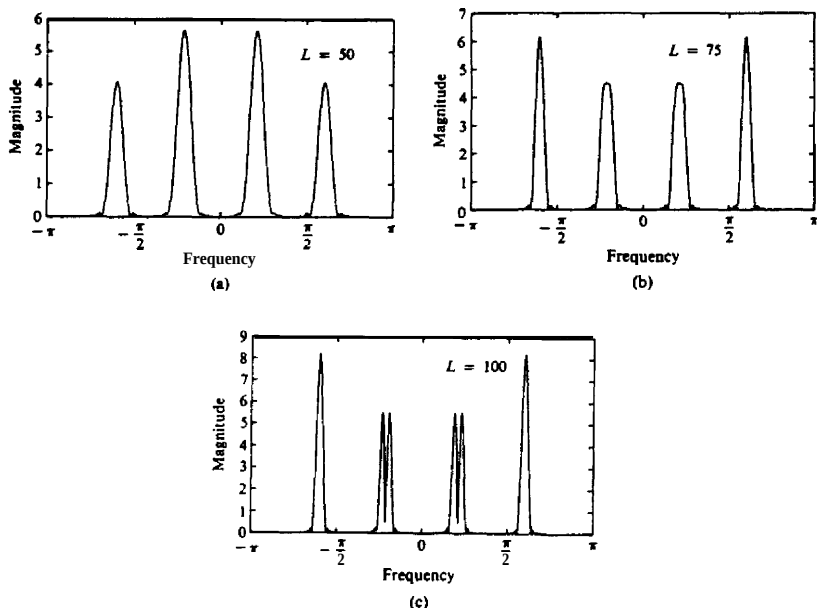


Figure 5.15 Magnitude spectrum of the signal in (5.4.8) as observed through a Hanning window.

$X(\omega)$, as would be the case when the number of samples L is **small**, the window spectrum **masks** the signal spectrum and, consequently, the DFT of the data reflects the spectral characteristics of the window function. Of course, this situation should be avoided.

Example 5.4.1

The exponential signal

$$x_a(t) = \begin{cases} e^{-t}, & t \geq 0 \\ 0, & t < 0 \end{cases}$$

is sampled at the rate $F_s = 20$ samples per second, and a block of 100 samples is used to estimate its spectrum. Determine the spectral characteristics of the signal $x_a(t)$ by computing the DFT of the finiteduration sequence. Compare the spectrum of the truncated discrete-time **signal** to the spectrum of the analog signal.

Solution The spectrum of the analog signal is

$$X_a(F) = \frac{1}{1 + j2\pi F}$$

The exponential analog signal sampled at the rate of 20 samples per **second** yields

the sequence

$$\begin{aligned} x(n) &= e^{-nT} = e^{-n/20}, & n \geq 0 \\ &= (e^{-1/20})^n = (0.95)^n, & n \geq 0 \end{aligned}$$

Now, let

$$x(n) = \begin{cases} (0.95)^n, & 0 \leq n \leq 99 \\ 0, & \text{otherwise} \end{cases}$$

The N-point DFT of the L = 100 point sequence is

$$\hat{X}(k) = \sum_{n=0}^{99} \hat{x}(n) e^{-j2\pi nk/N} \quad k = 0, 1, \dots, N-1$$

To obtain sufficient detail in the spectrum we choose N = 200. This is equivalent to padding the sequence $x(n)$ with 100 zeros.

The graph of the analog signal $x_a(t)$ and its magnitude spectrum $|X_a(F)|$ are illustrated in Fig. 5.16(a) and (b), respectively. The truncated sequence $x(n)$ and its N = 200 point DFT (magnitude) are illustrated in Fig. 5.16(c) and (d), respectively.

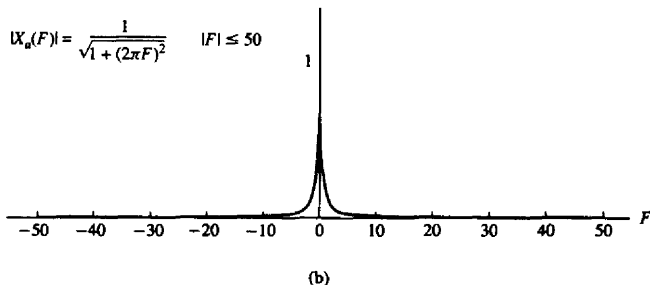
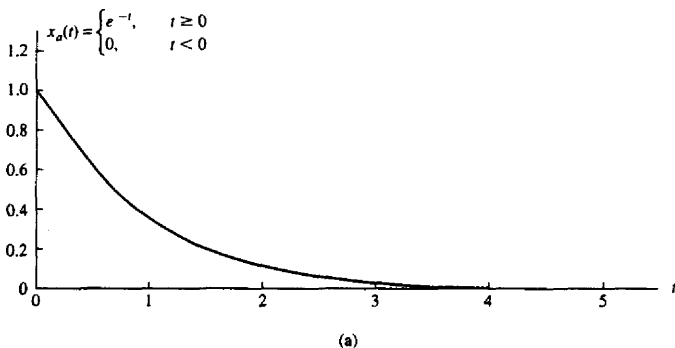
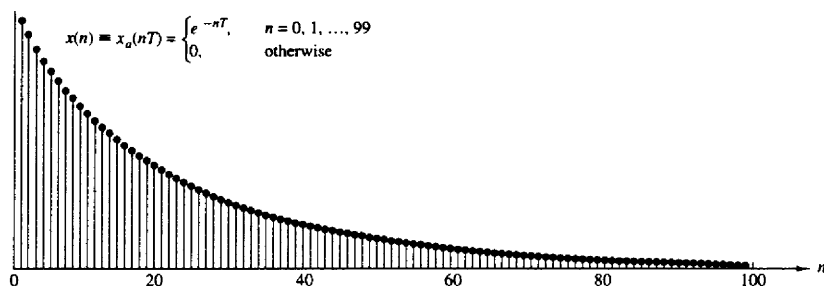
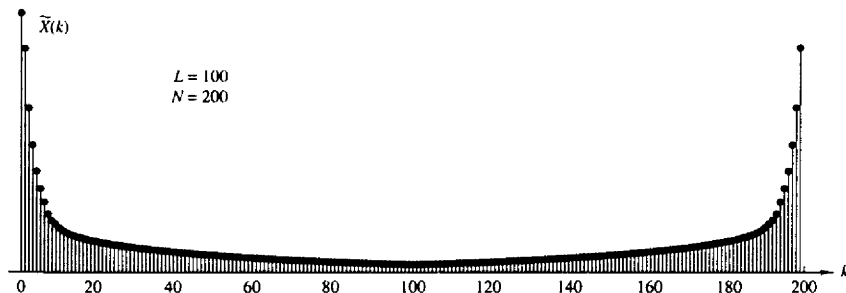


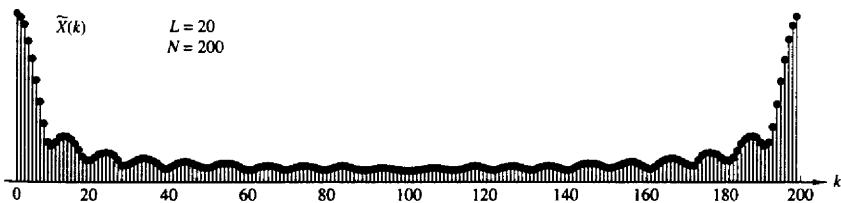
Figure 5.16 Effect of windowing (truncating) the sampled version of the analog signal in Example 5.4.1.



(c)



(d)



(e)

Figure 116 Continued

In this case the DFT $\{X(k)\}$ bears a close resemblance to the spectrum of the analog signal. The effect of the window function is relatively small.

On the other hand, suppose that a window function of length $L = 20$ is selected. Then the truncated sequence $x(n)$ is now given as

$$\hat{x}(n) = \begin{cases} (0.95)^n, & 0 \leq n \leq 19 \\ 0, & \text{otherwise} \end{cases}$$

Its $N = 200$ point DFT is illustrated in Fig. 5.16(e). Now the effect of the wider spectral window function is clearly evident. First, the main peak is very wide as a result of the wide spectral window. Second, the sinusoidal envelope variations in the spectrum away from the main peak are due to the large sidelobes of the rectangular window spectrum. Consequently, the DFT is no longer a good approximation of the analog signal spectrum.

5.5 SUMMARY AND REFERENCES

The major focus of this chapter was on the discrete Fourier transform, its properties and its applications. We developed the DFT by sampling the spectrum $X(\omega)$ of the sequence $x(n)$.

Frequency-domain sampling of the spectrum of a discrete-time signal is particularly important in the processing of digital signals. Of particular significance is the **DFT**, which was shown to uniquely represent a finite-duration sequence in the frequency domain. The existence of computationally efficient algorithms for the DFT, which are described in Chapter 6, make it possible to digitally process signals in the frequency domain much faster than in the time domain. The processing methods in which the **DFT** is especially suitable include linear filtering as described in this chapter and correlation, and spectrum analysis, which are treated in Chapters 6 and 12. A particularly lucid and concise treatment of the **DFT** and its application to frequency analysis is given in the book by **Brigham** (1988).

PROBLEMS

- 5.1 The first five points of the eight-point DFT of a real-valued sequence are $(0.25, 0.125 - j0.3018, 0, 0.125 - j0.0518, 0)$. Determine the remaining three points.
- 5.2 Compute the eight-point circular convolution for the following sequences.
- (a) $x_1(n) = \{1, 1, 1, 1, 0, 0, 0, 0\}$
 $x_2(n) = \sin \frac{3\pi}{8}n \quad 0 \leq n \leq 7$
- (b) $x_1(n) = (\frac{1}{4})^n \quad 0 \leq n \leq 7$
 $x_2(n) = \cos \frac{3\pi}{8}n \quad 0 \leq n \leq 7$
- (c) Compute the DFT of the two circular convolution sequences using the DFTs of $x_1(n)$ and $x_2(n)$.
- 5.3 Let $X(k), 0 \leq k \leq N-1$, be the N -point DFT of the sequence $x(n), 0 \leq n \leq N-1$. We define

$$\hat{X}(k) = \begin{cases} X(k), & 0 \leq k \leq k_c, N - k_c \leq k \leq N - 1 \\ 0, & k_c < k < N - k_c \end{cases}$$

and we compute the inverse N -point DFT of $\hat{X}(k), 0 \leq k \leq N-1$. What is the effect of this process on the sequence $x(n)$? Explain.

5.4 For the sequences

$$x_1(n) = \cos \frac{2\pi}{N}n \quad x_2(n) = \sin \frac{2\pi}{N}n \quad 0 \leq n \leq N-1$$

determine the N-point:

- (a) Circular convolution $x_1(n) \circledast x_2(n)$
- (b) Circular correlation of $x_1(n)$ and $x_2(n)$
- (c) Circular autocorrelation of $x_1(n)$
- (d) Circular autocorrelation of $x_2(n)$

5.5 Compute the quantity

$$\sum_{n=0}^{N-1} x_1(n)x_2(n)$$

for the following pairs of sequences.

- (a) $x_1(n) = x_2(n) = \cos \frac{2\pi}{N}n \quad 0 \leq n \leq N-1$
- (b) $x_1(n) = \cos \frac{2\pi}{N}n \quad x_2(n) = \sin \frac{2\pi}{N}n \quad 0 \leq n \leq N-1$
- (c) $x_1(n) = \delta(n) + \delta(n-8) \quad x_2(n) = u(n) - u(n-N)$

5.6 Determine the N-point DFT of the Blackman window

$$w(n) = 0.42 - 0.5 \cos \frac{2\pi n}{N-1} + 0.08 \cos \frac{4\pi n}{N-1} \quad 0 \leq n \leq N-1$$

5.7 If $X(k)$ is the DFT of the sequence $x(n)$, determine the N-point DFTs of the sequences

$$x_c(n) = x(n) \cos \frac{2\pi kn}{N} \quad 0 \leq n \leq N-1$$

and

$$x_s(n) = x(n) \sin \frac{2\pi kn}{N} \quad 0 \leq n \leq N-1$$

in terms of $X(k)$.

5.8 Determine the circular convolution of the sequences

$$x_1(n) = \{1, 2, 3, 1\}$$

↑

$$x_2(n) = \{4, 3, 2, 2\}$$

↑

using the timedomain formula in (5.2.39).

5.9 Use the four-point DFT and IDFT to determine the sequence

$$x_3(n) = x_1(n) \circledast x_2(n)$$

where $x_1(n)$ and $x_2(n)$ are the sequence given in Problem 5.8.

5.10 Compute the energy of the N-point sequence

$$x(n) = \cos \frac{2\pi kn}{N} \quad 0 \leq n \leq N-1$$

5.11 Given the eight-point DFT of the sequence

$$x(n) = \begin{cases} 1, & 0 \leq n \leq 3 \\ 0, & 4 \leq n \leq 7 \end{cases}$$

compute the DFT of the sequences:

$$(a) \quad x_1(n) = \begin{cases} 1, & n = 0 \\ 0, & 1 \leq n \leq 4 \\ 1, & 5 \leq n \leq 7 \end{cases}$$

$$(b) \quad x_2(n) = \begin{cases} 0, & 0 \leq n \leq 1 \\ 1, & 2 \leq n \leq 5 \\ 0, & 6 \leq n \leq 7 \end{cases}$$

5.12 Consider a finite-duration sequence

$$x(n) = \{0, 1, 2, 3, 4\}$$

↑

(a) Sketch the sequence $s(n)$ with six-point DFT

$$S(k) = W_6^* X(k) \quad k = 0, 1, \dots, 6$$

(b) Determine the sequence $y(n)$ with six-point DFT $Y(k) = \operatorname{Re}\{X(k)\}$.

(c) Determine the sequence $v(n)$ with six-point DFT $V(k) = \operatorname{Im}\{X(k)\}$.

5.W Let $x_p(n)$ be a periodic sequence with fundamental period N . Consider the following DFTs:

$$x_p(n) \xrightarrow[N]{\text{DFT}} X_1(k)$$

$$x_p(n) \xrightarrow[3N]{\text{DFT}} X_3(k)$$

(a) What is the relationship between $X_1(k)$ and $X_3(k)$?

(b) Verify the result in part (a) using the sequence

$$x_p(n) = \{\dots 1, 2, 1, 2, 1, 2, 1, 2, \dots\}$$

↑

5.14 Consider the sequences

$$x_1(n) = \{0, 1, 2, 3, 4\}$$

↑

$$x_2(n) = \{0, 1, 0, 0, 0\}$$

↑

$$s(n) = \{1, 0, 0, 0, 0\}$$

↑

and their 5-point DFTs.

(a) Determine a sequence $y(n)$ so that $Y(k) = X_1(k)X_2(k)$.

(b) Is there a sequence $x_3(n)$ such that $S(k) = X_1(k)X_3(k)$?

5.15 Consider a causal LTI system with system function

$$H(z) = \frac{1}{1 - 0.5z^{-1}}$$

The output $y(n]$ of the system is known for $0 \leq n \leq 63$. Assuming that $H(z)$ is available, can you develop a 64-point DFT method to recover the sequence $x(n)$, $0 \leq n \leq 63$? Can you recover all values of $x(n)$ in this interval?

5.16* The impulse response of an LTI system is given by $h(n) = \delta(n) - \frac{1}{4}\delta(n - k_0)$. To determine the impulse response $g(n)$ of the inverse system, an engineer computes the N -point DFT $H(k)$, $N = 4k_0$, of $h(n)$ and then defines $g(n)$ as the inverse DFT of

$G(k) = 1/H(k)$, $k = 0, 1, 2, \dots, N-1$. Determine $g(n)$ and the convolution $h(n) * g(n)$, and comment on whether the system with impulse response $g(n)$ is the inverse of the system with impulse response $h(n)$.

- 5.17* Determine the eight-point DFT of the signal

$$x(n) = [1, 1, 1, 1, 1, 1, 0, 0]$$

and sketch its magnitude and phase.

- 5.18 A linear time-invariant system with frequency response $H(\omega)$ is excited with the periodic input

$$x(n) = \sum_{k=-\infty}^{\infty} \delta(n - kN)$$

Suppose that we compute the N -point DFT $Y(k)$ of the samples $y(n)$, $0 \leq n \leq N-1$ of the output sequence. How is $Y(k)$ related to $H(\omega)$?

- 5.19 DFT of real sequences with special symmetries

- (a) Using the symmetry properties of Section 5.2 (especially the decomposition properties), explain how we can compute the DFT of two real symmetric (even) and two real antisymmetric (odd) sequences simultaneously using an N -point DFT only.
- (b) Suppose now that we are given four real sequences $x_i(n)$, $i = 1, 2, 3, 4$, that are all symmetric [i.e., $x_i(n) = x_i(N-n)$, $0 \leq n \leq N-1$]. Show that the sequences

$$s_i(n) = x_i(n+1) - x_i(n-1)$$

are antisymmetric [i.e., $s_i(n) = -s_i(N-n)$ and $s_i(0) = 0$].

- (c) Form a sequence $x(n)$ using $x_1(n)$, $x_2(n)$, $s_3(n)$, and $s_4(n)$ and show how to compute the DFT $X_i(k)$ of $x_i(n)$, $i = 1, 2, 3, 4$ from the N -point DFT $X(k)$ of $x(n)$.
- (d) Are there any frequency samples of $X_i(k)$ that cannot be recovered from $X(k)$? Explain.

- 5.20 DFT of real sequences with odd harmonics only Let $x(n)$ be an N -point real sequence with N -point DFT $X(k)$ (N even). In addition, $x(n)$ satisfies the following symmetry property:

$$x\left(n + \frac{N}{2}\right) = -x(n) \quad n = 0, 1, \dots, \frac{N}{2} - 1$$

that is, the upper half of the sequence is the negative of the lower half.

- (a) Show that

$$X(k) = 0 \quad k \text{ even}$$

that is, the sequence has a spectrum with odd harmonics.

- (b) Show that the values of this odd-harmonic spectrum can be computed by evaluating the $N/2$ -point DFT of a complex modulated version of the original sequence $x(n)$.
- 521 Let $x_a(t)$ be an analog signal with bandwidth $B = 3$ kHz. We wish to use a $N = 2^n$ -point DFT to compute the spectrum of the signal with a resolution less than or equal to 50 Hz. Determine (a) the minimum sampling rate, (b) the minimum number of required samples, and (c) the minimum length of the analog signal record.

5.22 Consider the **periodic** sequence

$$x_p(n) = \cos \frac{2\pi}{10}n \quad -\infty < n < \infty$$

with frequency $f_0 = \frac{1}{10}$ and fundamental period $N = 10$. **Determine the 10-point DFT of the sequence $x(n) = x_p(n)$, $0 \leq n \leq N-1$.**

5.23 Compute the N -point DFTs of the signals

(a) $x(n) = \delta(n)$

(b) $x(n) = \delta(n - n_0) \quad 0 < n_0 < N$

(c) $x(n) = a^n \quad 0 \leq n \leq N-1$

(d) $x(n) = \begin{cases} 1, & 0 \leq n \leq N/2 - 1 \text{ (} N \text{ even)} \\ 0, & N/2 \leq n \leq N-1 \end{cases}$

(e) $x(n) = e^{j(2\pi/N)k_0 n} \quad 0 \leq n \leq N-1$

(f) $x(n) = \cos \frac{2\pi}{N} k_0 n \quad 0 \leq n \leq N-1$

(g) $x(n) = \sin \frac{2\pi}{N} k_0 n \quad 0 \leq n \leq N-1$

(h) $x(n) = \begin{cases} 1, & n \text{ even} \\ 0, & n \text{ odd} \end{cases} \quad 0 \leq n \leq N-1$

5.24 Consider the finite-duration signal

$$x(n) = \{1, 2, 3, 1\}$$

(a) Compute its four-point DFT by solving explicitly the 4-by-4 system of linear equations defined by the inverse DFT formula.

(b) Check the answer in part (a) by computing the four-point DFT, using its definition.

5.25 (a) Determine the Fourier **transform** $X(\omega)$ of the signal

$$x(n) = \{1, 2, 3, 2, 1, 0\}$$

↑

(b) Compute the 6-point DFT $V(k)$ of the signal

$$v(n) = \{3, 2, 1, 0, 1, 2\}$$

(c) Is there any relation between $X(\omega)$ and $V(k)$? **Explain.**

5.26 Prove the identity

$$\sum_{l=-\infty}^{\infty} \delta(n + lN) = \frac{1}{N} \sum_{k=0}^{N-1} e^{j(2\pi/N)kn}$$

(Hint: Find the DFT of the periodic signal in the left-hand side.)

5.27 **Computation of the even and odd harmonics using the DFT** Let $x(n)$ be an N -point sequence with an N -point DFT $X(k)$ (N even)

(a) Consider the **time-aliased** sequence

$$y(n) = \begin{cases} \sum_{l=-\infty}^{\infty} x(n + lM), & 0 \leq n \leq M-1 \\ 0, & \text{elsewhere} \end{cases}$$

What is the relationship between the M -point DFT $Y(k)$ of $y(n)$ and the Fourier transform $X(\omega)$ of $x(n)$?

(b) Let

$$y(n) = \begin{cases} x(n) + x\left(n + \frac{N}{2}\right), & 0 \leq n \leq N-1 \\ 0, & \text{elsewhere} \end{cases}$$

and

$$y(n) \xleftrightarrow[N/2]{\text{DFT}} Y(k)$$

Show that $X(k) = Y(k/2)$, $k = 2, 4, \dots, N-2$.(c) Use the results in parts (a) and (b) to develop a procedure that computes the odd harmonics of $X(k)$ using an $N/2$ -point DFT.**5.28* Frequency-domain sampling** Consider the following discrete-time signal

$$x(n) = \begin{cases} a^{|n|} & |n| \leq L \\ 0, & |n| > L \end{cases}$$

where $a = 0.95$ and $L = 10$ (a) Compute and plot the signal $x(n)$.

(b) Show that

$$X(\omega) = \sum_{n=-\infty}^{\infty} x(n)e^{-j\omega n} = x(0) + 2 \sum_{n=1}^L x(n) \cos \omega n$$

Plot $X(\omega)$ by computing it at $\omega = \pi k/100$, $k = 0, 1, \dots, 100$.

(c) Compute

$$c_k = \frac{1}{N} X\left(\frac{2\pi}{N} K\right) \quad k = 0, 1, \dots, N-1$$

for $N = 30$.

(d) Determine and plot the signal

$$\tilde{x}(n) = \sum_{k=0}^{N-1} c_k e^{j(2\pi/N)kn}$$

What is the relation between the signals $x(n)$ and $\tilde{x}(n)$? Explain.(e) Compute and plot the signal $\tilde{x}_1(n) = \sum_{l=-\infty}^{\infty} x(n - lN)$, $-L \leq n \leq L$ for $N = 30$. Compare the signals $\tilde{x}(n)$ and $\tilde{x}_1(n)$.(f) Repeat parts (c) to (e) for $N = 15$.**5.29* Frequency-domain sampling** The signal $x(n) = a^{|n|}$, $-1 < a < 1$ has a Fourier transform

$$X(\omega) = \frac{1 - a^2}{1 - 2a \cos \omega + a^2}$$

(a) Plot $X(\omega)$ for $0 \leq \omega \leq 2\pi$, $a = 0.8$.Reconstruct and plot $X(\omega)$ from its samples $X(2\pi k/N)$, $0 \leq k \leq N-1$ for:(b) $N = 20$ (c) $N = 100$ (d) Compare the spectra obtained in parts (b) and (c) with the original spectrum $X(\omega)$ and explain the differences.(e) Illustrate the **time-domain** aliasing when $N = 20$.

5.30* *Frequency analysis of amplitude-modulated discrete-time signal* The discrete-time signal

$$x(n) = \cos 2\pi f_1 n + \cos 2\pi f_2 n$$

where $f_1 = \frac{1}{18}$ and $f_2 = \frac{5}{128}$, modulates the amplitude of the carrier

$$x_c(n) = \cos 2\pi f_c n$$

where $f_c = \frac{50}{128}$. The resulting amplitude-modulated signal is

$$x_{am}(n) = x(n) \cos 2\pi f_c n$$

- Sketch the signals $x(n)$, $x_c(n)$, and $x_{am}(n)$, $0 \leq n \leq 255$.
 - Compute and sketch the 128-point DFT of the signal $x_{am}(n)$, $0 \leq n \leq 127$.
 - Compute and sketch the 128-point DFT of the signal $x_{am}(n)$, $0 \leq n \leq 99$.
 - Compute and sketch the 256-point DFT of the signal $x_{am}(n)$, $0 \leq n \leq 179$.
 - Explain the results obtained in parts (b) through (d), by deriving the spectrum of the amplitude-modulated signal and comparing it with the experimental results.
- 5.31*** The sawtooth waveform in Fig. P5.31 can be expressed in the form of a Fourier series as

$$x(t) = \frac{2}{\pi} \left(\sin \pi t - \frac{1}{2} \sin 2\pi t + \frac{1}{3} \sin 3\pi t - \frac{1}{4} \sin 4\pi t \dots \right)$$

- Determine the Fourier series coefficients c_k .
- Use an N-point subroutine to generate samples of this signal in the time domain using the first six terms of the expansion for $N \approx 64$ and $N = 128$. Plot the signal $x(t)$ and the samples generated, and comment on the results.

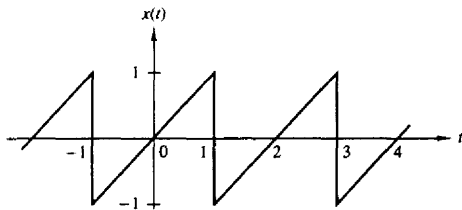


Figure P5.31

5.32 Recall that the Fourier transform of $x(t) = e^{j\Omega_0 t}$ is $X(j\Omega) = 2\pi \delta(\Omega - \Omega_0)$ and the Fourier transform of

$$p(t) = \begin{cases} 1, & 0 \leq t \leq T_0 \\ 0, & \text{otherwise} \end{cases}$$

is

$$P(j\Omega) = T_0 \frac{\sin \Omega T_0 / 2}{\Omega T_0 / 2} e^{-j\Omega T_0 / 2}$$

- Determine the Fourier transform $Y(j\Omega)$ of

$$y(t) = p(t)e^{j\Omega_0 t}$$

and roughly sketch $|Y(j\Omega)|$ versus Ω .

- (b) Now consider the exponential sequence

$$x(n) = e^{j\omega_0 n}$$

where ω_0 is some arbitrary frequency in the range $0 < \omega_0 < \pi$ radians. Give the most general condition that ω_0 must satisfy in order for $x(n)$ to be periodic with period P (P is a positive integer).

- (c) Let
- $y(n)$
- be the finite-duration sequence

$$y(n) = x(n)w_N(n) = e^{j\omega_0 n} w_N(n)$$

where $w_N(n)$ is a finite-duration rectangular sequence of length N and where $x(n)$ is not necessarily periodic. Determine $Y(\omega)$ and roughly sketch $|Y(\omega)|$ for $0 \leq \omega \leq 2\pi$. What effect does N have in $|Y(\omega)|$? Briefly comment on the similarities and differences between $|Y(\omega)|$ and $|Y(j\Omega)|$.

- (d) Suppose that

$$x(n) = e^{j(2\pi/P)n} \quad P \text{ a positive integer}$$

and

$$y(n) = w_N(n)x(n)$$

where $N = 1/P$, 1 a positive integer. Determine and sketch the N -point DFT of $y(n)$. Relate your answer to the characteristics of $|Y(\omega)|$.

- (e) Is the frequency sampling for the DFT in part (d) adequate for obtaining a rough approximation of $|Y(\omega)|$ directly from the magnitude of the DFT sequence $|Y(k)|$? If not, explain briefly how the sampling can be increased so that it will be possible to obtain a rough sketch of $|Y(\omega)|$ from an appropriate sequence $|Y(k)|$.



6

Efficient Computation of the DFT: Fast Fourier Transform Algorithms

As we have observed in the preceding chapter, the Discrete Fourier Transform (DFT) plays an important role in many applications of digital signal processing, including linear filtering, correlation analysis, and spectrum analysis. A major reason for its importance is the existence of efficient algorithms for computing the DFT.

The main topic of this chapter is the description of computationally efficient algorithms for evaluating the DFT. Two different approaches are described. One is a divide-and-conquer approach in which a DFT of size N , where N is a composite number, is reduced to the computation of smaller DFTs from which the larger DFT is computed. In particular, we present important computational algorithms, called fast Fourier transform (FFT) algorithms, for computing the DFT when the size N is a power of 2 and when it is a power of 4.

The second approach is based on the formulation of the DFT as a linear filtering operation on the data. This approach leads to two algorithms, the **Goertzel** algorithm and the **chirp-z transform** algorithm for computing the DFT via **linear** filtering of the data sequence.

6.1 EFFICIENT COMPUTATION OF THE DFT: FFT ALGORITHMS

In this section we present several methods for computing the DFT efficiently. In view of the importance of the DFT in various digital signal processing applications, such as linear filtering, correlation analysis, and spectrum analysis, its **efficient** computation is a topic that has received considerable attention by many mathematicians, engineers, and applied scientists.

Basically, the computational problem for the DFT is to compute the sequence $\{X(k)\}$ of N complex-valued numbers given another sequence of data $\{x(n)\}$ of

length N , according to the formula

$$X(k) = \sum_{n=0}^{N-1} x(n) W_N^{kn} \quad 0 \leq k \leq N-1 \quad (6.1.1)$$

where

$$W_N = e^{-j2\pi/N} \quad (6.1.2)$$

In general, the data sequence $x(n)$ is also assumed to be complex valued.

Similarly, the IDFT becomes

$$x(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k) W_N^{-nk} \quad 0 \leq n \leq N-1 \quad (6.1.3)$$

Since the **DFT** and IDFT involve basically the same type of computations, our discussion of efficient computational algorithms for the DFT applies as well to the efficient computation of the **IDFT**.

We observe that for each value of k , direct computation of $X(k)$ involves N complex multiplications ($4N$ real **multiplications**) and $N-1$ complex additions ($4N-2$ real additions). Consequently, to compute all N values of the **DFT** requires N^2 **complex** multiplications and $N^2 - N$ complex additions.

Direct computation of the DFT is basically inefficient primarily because it does not exploit the symmetry and periodicity properties of the phase factor W_N . In particular, these two properties are:

$$\text{Symmetry property: } W_N^{k+N/2} = -W_N^k \quad (6.1.4)$$

$$\text{Periodicity property: } W_N^{k+N} = W_N^k \quad (6.1.5)$$

The computationally efficient algorithms described in this section, known collectively as fast Fourier **transform** (FFT) algorithms, exploit these two basic properties of the phase factor.

6.1.1 Direct Computation of the DFT

For a complex-valued sequence $x(n)$ of N points, the DFT **may** be expressed as

$$X_R(k) = \sum_{n=0}^{N-1} \left[x_R(n) \cos \frac{2\pi kn}{N} + x_I(n) \sin \frac{2\pi kn}{N} \right] \quad (6.1.6)$$

$$X_I(k) = - \sum_{n=0}^{N-1} \left[x_R(n) \sin \frac{2\pi kn}{N} - x_I(n) \cos \frac{2\pi kn}{N} \right] \quad (6.1.7)$$

The direct computation of (6.1.6) and (6.1.7) requires:

1. $2N^2$ evaluations of trigonometric functions.
2. $4N^2$ real multiplications.

3. $4N(N-1)$ real additions.
4. A number of indexing and addressing operations.

These operations are typical of **DFT** computational algorithms. The operations in items 2 and 3 result in the DFT values $X_R(k)$ and $X_I(k)$. The indexing and addressing operations are necessary to fetch the data $x(n)$, $0 \leq n \leq N-1$, and the phase factors and to store the results. The variety of DFT algorithms **optimize** each of these computational processes in a different way.

6.1.2 Divide-and-Conquer Approach to Computation of the DFT

The development of computationally efficient algorithms for the DFT is made possible if we adopt a divide-and-conquer approach. This approach is based on the decomposition of an N -point DFT into successively smaller **DFTs**. This basic approach leads to a **family** of computationally efficient algorithms known collectively as **FFT** algorithms.

To illustrate the basic notions, let us consider the computation of an N -point DFT, where N can be factored as a product of two integers, that is.

$$N = LM \quad (6.1.8)$$

The assumption that N is not a prime number is not restrictive, since we can pad any sequence with zeros to ensure a factorization of the form (6.1.8).

Now the sequence $x(n)$, $0 \leq n \leq N-1$, can be stored in either a **one**-dimensional array indexed by n or as a two-dimensional array indexed by l and m , where $0 \leq l \leq L-1$ and $0 \leq m \leq M-1$ as illustrated in Fig. 6.1. Note that l is the row index and m is the column index. **Thus**, the sequence $x(n)$ can be stored in a rectangular array in a variety of ways, each of which depends on the mapping of index n to the indexes (l, m) .

For example, suppose that we select the mapping

$$n = Ml + m \quad (6.1.9)$$

This leads to an arrangement in which the first row consists of the first M elements of $x(n)$, the second row consists of the next M elements of $x(n)$, and so on, as illustrated in Fig. 6.2(a). On the other hand, the mapping

$$n = l + mL \quad (6.1.10)$$

stores the first L elements of $x(n)$ in the first column, the next L elements in the **second** column, and so on, as illustrated in Fig. 6.2(b).

A similar arrangement can be used to store the computed DFT values. In particular, the mapping is from the index k to a pair of indices (p, q) , where $0 \leq p \leq L-1$ and $0 \leq q \leq M-1$. If we select the mapping

$$k = Mp + q \quad (6.1.11)$$

Sec. 6.1 Efficient Computation of the DFT: FFT Algorithms

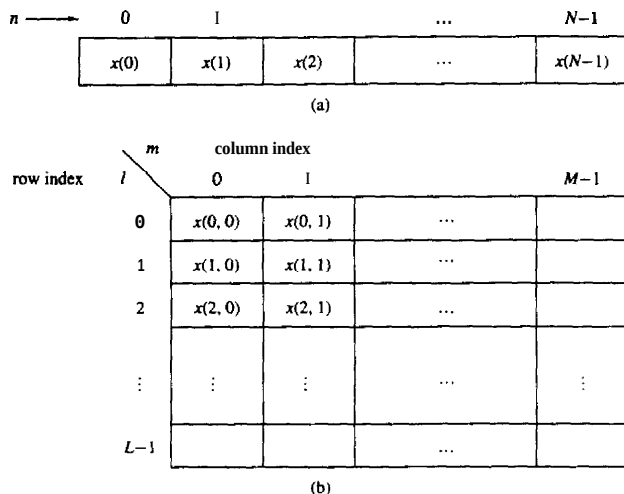


Figure 6.1 Two dimensional data array for storing the sequence $x(n)$, $0 \leq n \leq N-1$.

the DFT is stored on a row-wise basis, where the first row contains the first M elements of the DFT $X(k)$, the second row contains the next set of M elements, and so on. On the other hand, the mapping

$$k = qL + p \quad (6.1.12)$$

results in a column-wise storage of $X(k)$, where the first L elements are stored in the first column, the second set of L elements are stored in the second column, and so on.

Now suppose that $x(n)$ is mapped into the rectangular array $x(l, m)$ and $X(k)$ is mapped into a corresponding rectangular array $X(p, q)$. Then the DFT can be expressed as a double sum over the elements of the rectangular array multiplied by the corresponding phase factors. To be specific, let us adopt a column-wise mapping for $x(n)$ given by (6.1.10) and the row-wise mapping for the DFT given by (6.1.11). Then

$$X(p, q) = \sum_{m=0}^{M-1} \sum_{l=0}^{L-1} x(l, m) W_N^{(Mp+q)(mL+l)} \quad (6.1.13)$$

But

$$W_N^{(Mp+q)(mL+l)} = W_N^{MLmp} W_N^{MLq} W_N^{Mpl} W_N^{lq} \quad (6.1.14)$$

However, $W_N^{Nmp} = 1$, $W_N^{mqL} = W_{N/L}^{mq} = W_M^{mq}$, and $W_N^{Mpl} = W_{N/M}^{pl} = W_L^{pl}$.

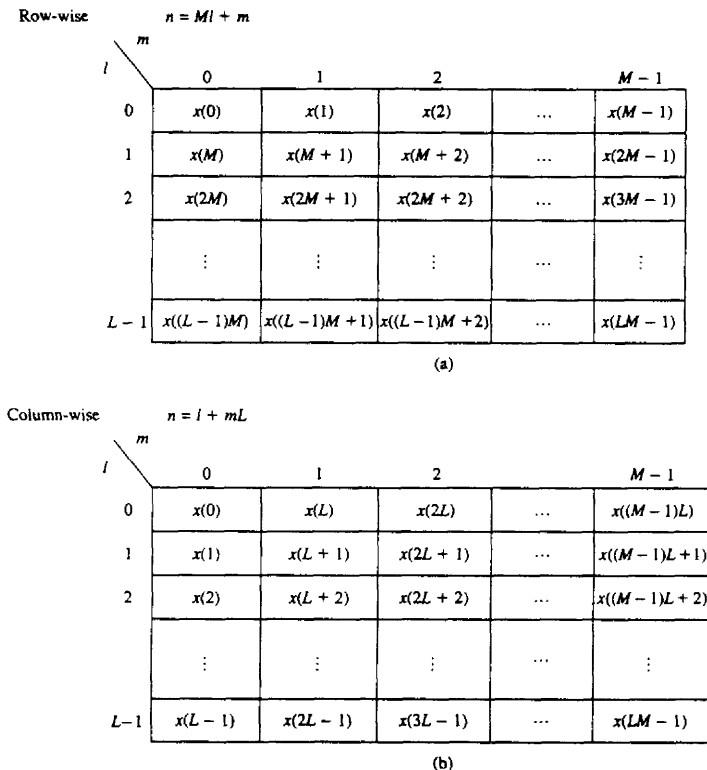


Figure 6.2 Two arrangements for the data arrays.

With these simplifications, (6.1.13) can be expressed as

$$X(p, q) = \sum_{l=0}^{L-1} \left\{ W_N^{lq} \left[\sum_{m=0}^{M-1} x(l, m) W_M^{mq} \right] \right\} W_L^{lp} \quad (6.1.15)$$

The expression in (6.1.15) involves the computation of DFTs of length M and length L . To elaborate, let us subdivide the computation into three steps:

L First, we compute the M -point DFTs

$$F(l, q) \equiv \sum_{m=0}^{M-1} x(l, m) W_M^{mq}, \quad 0 \leq q \leq M-1 \quad (6.1.16)$$

for each of the rows $l = 0, 1, \dots, L-1$.

2. Second, we compute a new rectangular array $G(l, q)$ defined as

$$G(l, q) = W_N^{lq} F(l, q) \quad \begin{matrix} 0 \leq l \leq L-1 \\ 0 \leq q \leq M-1 \end{matrix} \quad (6.1.17)$$

3. Finally, we compute the L -point DFTs

$$X(p, q) = \sum_{l=0}^{L-1} G(l, q) W_L^{lp} \quad (6.1.18)$$

for each column $q = 0, 1, \dots, M-1$, of the array $G(l, q)$.

On the surface it may appear that the computational procedure outlined above is more complex than the direct computation of the DFT. However, let us evaluate the computational complexity of (6.1.15). The first step involves the computation of L DFTs, each of M points. Hence this step requires LM^2 complex multiplications and $LM(M-1)$ complex additions. The second step requires LM complex multiplications. Finally, the third step in the computation requires ML^2 complex multiplications and $ML(L-1)$ complex additions. Therefore, the computational complexity is

$$\text{Complex multiplications: } N(M+L+1) \quad (6.1.19)$$

$$\text{Complex additions: } N(M+L-2)$$

where $N = ML$. Thus the number of multiplications has been reduced from N^2 to $N(M+L+1)$ and the number of additions has been reduced from $N(N-1)$ to $N(M+L-2)$.

For example, suppose that $N = 1000$ and we select $L = 2$ and $M = 500$. Then, instead of having to perform 10^6 complex multiplications via direct computation of the DFT, this approach leads to 503,000 complex multiplications. This represents a reduction by approximately a factor of 2. The number of additions is also reduced by about a factor of 2.

When N is a highly composite number, that is, N can be factored into a product of prime numbers of the form

$$N = r_1 r_2 \cdots r_v \quad (6.1.20)$$

then the decomposition above can be repeated $(v-1)$ more times. This procedure results in smaller DFTs, which, in turn, leads to a more efficient computational algorithm.

In effect, the first segmentation of the sequence $x(n)$ into a rectangular array of M columns with L elements in each column resulted in DFTs of sizes L and M . Further decomposition of the data in effect involves the segmentation of each row (or column) into smaller rectangular arrays which result in smaller DFTs. This procedure terminates when N is factored into its prime factors.

Example 6.1.1

To illustrate this computational procedure, let us consider the computation of an $N = 15$ point DFT. Since $N = 5 \times 3 = 15$, we select $L = 5$ and $M = 3$. In other

words, we store the 15-point sequence $x(n)$ column-wise as follows:

Row 1:	$x(0, 0) = x(0)$	$x(0, 1) = x(5)$	$x(0, 2) = x(10)$
Row 2:	$x(1, 0) = x(1)$	$x(1, 1) = x(6)$	$x(1, 2) = x(11)$
Row 3:	$x(2, 0) = x(2)$	$x(2, 1) = x(7)$	$x(2, 2) = x(12)$
Row 4:	$x(3, 0) = x(3)$	$x(3, 1) = x(8)$	$x(3, 2) = x(13)$
Row 5:	$x(4, 0) = x(4)$	$x(4, 1) = x(9)$	$x(4, 2) = x(14)$

Now, we compute the three-point DFTs for each of the five rows. This leads to the following 5×3 array:

$F(0, 0)$	$F(0, 1)$	$F(0, 2)$
$F(1, 0)$	$F(1, 1)$	$F(1, 2)$
$F(2, 0)$	$F(2, 1)$	$F(2, 2)$
$F(3, 0)$	$F(3, 1)$	$F(3, 2)$
$F(4, 0)$	$F(4, 1)$	$F(4, 2)$

The next step is to multiply each of the terms $F(l, q)$ by the *phase factors* $W_N^{lq} = W_{15}^{lq}$, $0 \leq l \leq 4$ and $0 \leq q \leq 2$. This computation results in the 5×3 array:

Column 1	Column 2	Column 3
$G(0, 0)$	$G(0, 1)$	$G(0, 2)$
$G(1, 0)$	$G(1, 1)$	$G(1, 2)$
$G(2, 0)$	$G(2, 1)$	$G(2, 2)$
$G(3, 0)$	$G(3, 1)$	$G(3, 2)$
$G(4, 0)$	$G(4, 1)$	$G(4, 2)$

The final step is to compute the five-point DFTs for each of the three columns. This computation yields the desired values of the DFT in the form

$X(0, 0) = X(0)$	$X(0, 1) = X(1)$	$X(0, 2) = X(2)$
$X(1, 0) = X(3)$	$X(1, 1) = X(4)$	$X(1, 2) = X(5)$
$X(2, 0) = X(6)$	$X(2, 1) = X(7)$	$X(2, 2) = X(8)$
$X(3, 0) = X(9)$	$X(3, 1) = X(10)$	$X(3, 2) = X(11)$
$X(4, 0) = X(12)$	$X(4, 1) = X(13)$	$X(4, 2) = X(14)$

Figure 6.3 illustrates the steps in the computation.

It is interesting to view the segmented data sequence and the resulting DFT in terms of one-dimensional arrays. When the input sequence $x(n)$ and the output DFT $X(k)$ in the two-dimensional arrays are read across from row 1 through row 5, we obtain the following sequences:

INPUT ARRAY

$x(0) \ x(5) \ x(10) \ x(1) \ x(6) \ x(11) \ x(2) \ x(7) \ x(12) \ x(3) \ x(8) \ x(13) \ x(4) \ x(9) \ x(14)$

OUTPUT ARRAY

$X(0) \ X(1) \ X(2) \ X(3) \ X(4) \ X(5) \ X(6) \ X(7) \ X(8) \ X(9) \ X(10) \ X(11) \ X(12) \ X(13) \ X(14)$

We observe that the input data sequence is shuffled from the normal order in the computation of the DFT. On the other hand, the output sequence occurs in normal order. In this case the rearrangement of the input data array is due to the

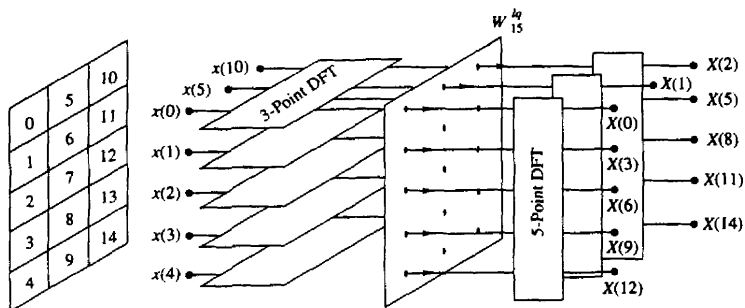


Figure 6.3 Computation of $N = 15$ -point DFT by means of 3-point and 5-point DFTs.

segmentation of the one-dimensional array into a rectangular array and the order in which the DFTs are computed. This shuffling of either the input data sequence or the output DFT sequence is a characteristic of most FFT algorithms.

To summarize, the algorithm that we **have** introduced involves the following computations:

Algorithm 1

1. Store the signal column-wise.
2. Compute the M -point DFT of each row.
3. Multiply the resulting array by the phase factors W_N^{lq} .
4. Compute the L -point DFT of each column
5. Read the resulting array row-wise.

An additional algorithm with a similar computational structure can be obtained if the input signal is stored row-wise and the resulting transformation is column-wise. In this **case** we select as

$$\begin{aligned} n &= Ml + m \\ k &= qL + p \end{aligned} \quad (6.1.21)$$

This choice of indices leads to the **formula** for the DFT in the **form**

$$\begin{aligned} X(p, q) &= \sum_{m=0}^{M-1} \sum_{l=0}^{L-1} x(l, m) W_N^{pm} W_L^{pl} W_M^{qm} \\ &= \sum_{m=0}^{M-1} W_M^{mq} \left[\sum_{l=0}^{L-1} x(l, m) W_L^{lp} \right] W_N^{mp} \end{aligned} \quad (6.1.22)$$

Thus we obtain a second algorithm.

Algorithm 2

1. Store the signal row-wise.
2. Compute the L-point DFT at each column.
3. Multiply the resulting array by the factors W_N^{pm} .
4. Compute the M-point DFT of each row.
5. Read the resulting array column-wise.

The two algorithms given above have the same complexity. However, they differ in the arrangement of the computations. In the following sections we exploit the divide-and-conquer approach to derive fast algorithms when the size of the DFT is restricted to be a power of 2 or a power of 4.

6.1.3 Radix-2 FFT Algorithms

In the preceding section we described four algorithms for efficient computation of the DFT based on the divide-and-conquer approach. Such an approach is applicable when the number N of data points is not a prime. In particular, the approach is very efficient when N is highly composite, that is, when N can be factored as $N = r_1 r_2 r_3 \cdots r_v$, where the $\{r_j\}$ are prime.

Of particular importance is the case in which $r_1 = r_2 = \cdots = r_v \equiv r$, so that $N = r^v$. In such a case the DFTs are of size r , so that the computation of the N -point DFT has a regular pattern. The number r is called the radix of the FFT algorithm.

In this section we describe radix-2 algorithms, which are by far the most widely used FFT algorithms. Radix-4 algorithms are described in the following section.

Let us consider the computation of the $N = 2^v$ point DFT by the divide-and-conquer approach specified by (6.1.16) through (6.1.18). We select $M = N/2$ and $L = 2$. This selection results in a split of the N -point data sequence into two $N/2$ -point data sequences $f_1(n)$ and $f_2(n)$, corresponding to the even-numbered and odd-numbered samples of $x(n)$, respectively, that is,

$$\begin{aligned} f_1(n) &= x(2n) \\ f_2(n) &= x(2n+1), \quad n = 0, 1, \dots, \frac{N}{2} - 1 \end{aligned} \quad (6.1.23)$$

Thus $f_1(n)$ and $f_2(n)$ are obtained by decimating $x(n)$ by a factor of 2, and hence the resulting FFT algorithm is called a decimation-in-time algorithm.

Now the N -point DFT can be expressed in terms of the DFTs of the decimated sequences as follows:

$$X(k) = \sum_{n=0}^{N-1} x(n) W_N^{kn} \quad k = 0, 1, \dots, N-1$$

$$\begin{aligned}
&= \sum_{n \text{ even}} x(n) W_N^{kn} + \sum_{n \text{ odd}} x(n) W_N^{kn} \\
&= \sum_{m=0}^{(N/2)-1} x(2m) W_N^{2mk} + \sum_{m=0}^{(N/2)-1} x(2m+1) W_N^{k(2m+1)}
\end{aligned} \quad (6.1.24)$$

But $W_N^2 = W_{N/2}$. With this substitution, (6.1.24) can be expressed as

$$\begin{aligned}
X(k) &= \sum_{m=0}^{(N/2)-1} f_1(m) W_{N/2}^{km} + W_N^k \sum_{m=0}^{(N/2)-1} f_2(m) W_{N/2}^{km} \\
&= F_1(k) + W_N^k F_2(k) \quad k = 0, 1, \dots, N-1
\end{aligned} \quad (6.1.25)$$

where $F_1(k)$ and $F_2(k)$ are the $N/2$ -point DFTs of the sequences $f_1(m)$ and $f_2(m)$, respectively.

Since $F_1(k)$ and $F_2(k)$ are periodic, with period $N/2$, we have $F_1(k + N/2) = F_1(k)$ and $F_2(k + N/2) = F_2(k)$. In addition, the factor $W_N^{k+N/2} = -W_N^k$. Hence (6.1.25) can be expressed as

$$X(k) = F_1(k) + W_N^k F_2(k) \quad k = 0, 1, \dots, \frac{N}{2} - 1 \quad (6.1.26)$$

$$X\left(k + \frac{N}{2}\right) = F_1(k) - W_N^k F_2(k) \quad k = 0, 1, \dots, \frac{N}{2} - 1 \quad (6.1.27)$$

We observe that the direct computation of $F_1(k)$ requires $(N/2)^2$ complex multiplications. The same applies to the computation of $F_2(k)$. Furthermore, there are $N/2$ additional complex multiplications required to compute $W_N^k F_2(k)$. Hence the computation of $X(k)$ requires $2(N/2)^2 + N/2 = N^2/2 + N/2$ complex multiplications. This first step results in a reduction of the number of multiplications from N^2 to $N^2/2 + N/2$, which is about a factor of 2 for N large.

To be consistent with our previous notation, we may define

$$G_1(k) = F_1(k) \quad k = 0, 1, \dots, \frac{N}{2} - 1$$

$$G_2(k) = W_N^k F_2(k) \quad k = 0, 1, \dots, \frac{N}{2} - 1$$

Then the DFT $X(k)$ may be expressed as

$$\begin{aligned}
X(k) &= G_1(k) + G_2(k) \quad k = 0, 1, \dots, \frac{N}{2} - 1 \\
X\left(k + \frac{N}{2}\right) &= G_1(k) - G_2(k) \quad k = 0, 1, \dots, \frac{N}{2} - 1
\end{aligned} \quad (6.1.28)$$

This computation is illustrated in Fig. 6.4.

Having performed the decimation-in-time **once**, we can repeat the process for each of the sequences $f_1(n)$ and $f_2(n)$. Thus $f_1(n)$ would result in the two

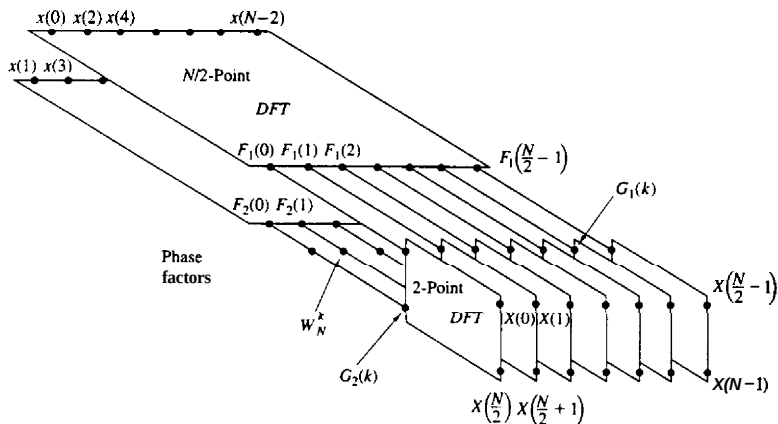


Figure 6.4 First step in the decimation-in-time algorithm.

$N/4$ -point sequences

$$\begin{aligned} v_{11}(n) &= f_1(2n) & n = 0, 1, \dots, \frac{N}{4} - 1 \\ v_{12}(n) &= f_1(2n + 1) & n = 0, 1, \dots, \frac{N}{4} - 1 \end{aligned} \quad (6.1.29)$$

and $f_2(n)$ would yield

$$\begin{aligned} v_{21}(n) &= f_2(2n) & n = 0, 1, \dots, \frac{N}{4} - 1 \\ v_{22}(n) &= f_2(2n + 1) & n = 0, 1, \dots, \frac{N}{4} - 1 \end{aligned} \quad (6.1.30)$$

By computing $N/4$ -point DFTs, we would obtain the $N/2$ -point DFTs $F_1(k)$ and $F_2(k)$ from the relations

$$\begin{aligned} F_1(k) &= V_{11}(k) + W_{N/2}^k V_{12}(k) & k = 0, 1, \dots, \frac{N}{4} - 1 \\ F_1\left(k + \frac{N}{4}\right) &= V_{11}(k) - W_{N/2}^k V_{12}(k) & k = 0, 1, \dots, \frac{N}{4} - 1 \end{aligned} \quad (6.1.31)$$

$$\begin{aligned} F_2(k) &= V_{21}(k) + W_{N/2}^k V_{22}(k) & k = 0, 1, \dots, \frac{N}{4} - 1 \\ F_2\left(k + \frac{N}{4}\right) &= V_{21}(k) - W_{N/2}^k V_{22}(k) & k = 0, \dots, \frac{N}{4} - 1 \end{aligned} \quad (6.1.32)$$

where the $\{V_{ij}(k)\}$ are the $N/4$ -point DFTs of the sequences $\{v_{ij}(n)\}$.

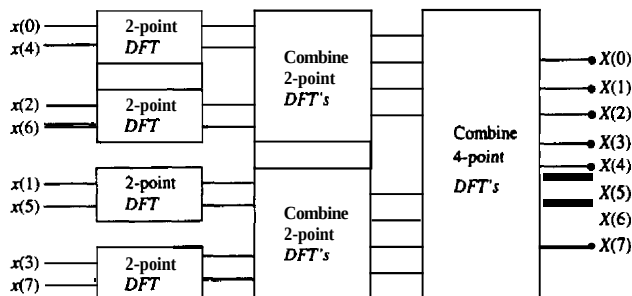
TABLE 6.1 COMPARISON OF COMPUTATIONAL COMPLEXITY FOR THE DIRECT COMPUTATION OF THE OFT VERSUS THE FFT ALGORITHM

Number of Points, N	Complex Multiplications in Direct Computation, N^2	Complex Multiplications in FFT Algorithm, $(N/2) \log_2 N$	Speed Improvement Factor
4	16	4	4.0
8	64	12	5.3
16	256	32	8.0
32	1,024	80	12.8
64	4,096	192	21.3
128	16,384	448	36.6
256	65,536	1,024	64.0
512	262,144	2,304	113.8
1,024	1,048,576	5,120	204.8

We observe that the computation of $\{V_{ij}(k)\}$ requires $4(N/4)^2$ multiplications and hence the computation of $F_1(k)$ and $F_2(k)$ can be accomplished with $N^2/4 + N/2$ complex multiplications. An additional $N/2$ complex multiplications are required to compute $X(k)$ from $F_1(k)$ and $F_2(k)$. Consequently, the total number of multiplications is reduced approximately by a factor of 2 again to $N^2/4 + N$.

The decimation of the data sequence can be repeated again and again until the resulting sequences are reduced to one-point sequences. For $N = 2^v$, this decimation can be performed $v = \log_2 N$ times. Thus the total number of complex multiplications is reduced to $(N/2) \log_2 N$. The number of complex additions is $N \log_2 N$. Table 6.1 presents a comparison of the number of complex multiplications in the FFT and in the direct computation of the DFT.

For illustrative purposes, Fig. 6.5 depicts the computation of an $N = 8$ point DFT. We observe that the computation is performed in three stages, beginning with the computations of four two-point DFTs, then two four-point DFTs, and

Figure 6.5 Three stages in the computation of an $N = 8$ -point DFT

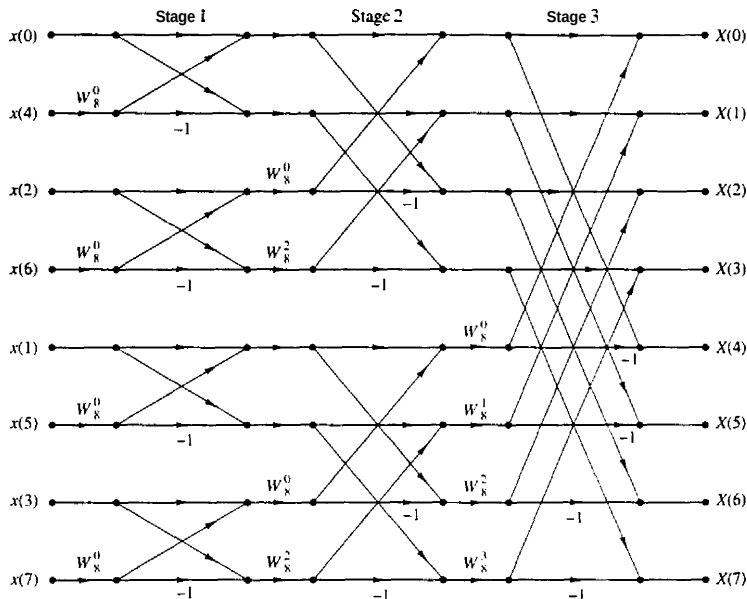


Figure 6.6 Eight-point decimation-in-time FFT algorithm.

finally, one eight-point DFT. The combination of the smaller DFTs to form the larger DFT is illustrated in Fig. 6.6 for $N = 8$.

Observe that the basic computation performed at every stage, as illustrated in Fig. 6.6, is to take two complex numbers, say the pair (a, b) , multiply b by W_N^r , and then add and subtract the product from a to form two new complex numbers (A, B) . This basic computation, which is shown in Fig. 6.7, is called a *butterfly* because the flow graph resembles a butterfly.

In general, each butterfly involves one complex multiplication and two complex additions. For $N = 2^v$, there are $N/2$ butterflies per stage of the computation process and $\log_2 N$ stages. Therefore, as previously indicated the total number of complex multiplications is $(N/2) \log_2 N$ and complex additions is $N \log_2 N$.

Once a butterfly operation is performed on a pair of complex numbers (a, b) to produce (A, B) , there is no need to save the input pair (a, b) . Hence we can

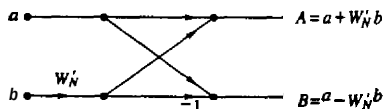


Figure 6.7 Basic butterfly computation in the decimation-in-time FFT algorithm.

store the result (A, B) in the same locations as (a, b) . Consequently, we require a fixed amount of storage, namely, $2N$ storage registers, in order to store the results (N complex **numbers**) of the computations at each stage. Since the **same** $2N$ storage locations are used throughout the computation of the N -point DFT, we say that the **computations** are **done** in place.

A second important observation is concerned with the order of the input data sequence after it is decimated $(v - 1)$ times. For example, if we consider the case where $N = 8$, we know that the first decimation yields the sequence $x(0), x(2), x(4), x(6), x(1), x(3), x(5), x(7)$, and the second decimation results in the sequence $x(0), x(4), x(2), x(6), x(1), x(5), x(3), x(7)$. This *shuffling* of the input data sequence has a well-defined order as can be ascertained from observing Fig. 6.8, which illustrates the decimation of the eight-point sequence. By expressing the index n , in the sequence $x(n)$, in binary form, we note that the order of the decimated data sequence is **easily** obtained by reading the binary representation of the index n in reverse order. Thus the data point $x(3) \equiv x(011)$ is placed in position $m = 110$ or $m = 6$ in the decimated array. Thus we say that the data $x(n)$ after decimation is stored in bit-reversed order.

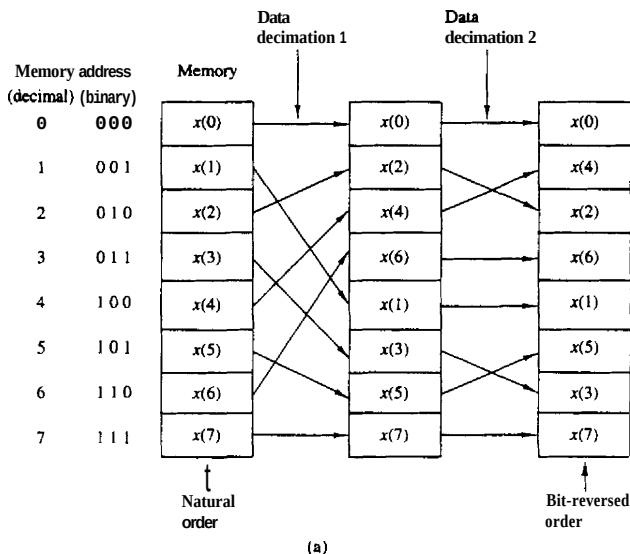
With the input data sequence stored in bit-reversed order and the butterfly computations performed in place, the resulting DFT sequence $X(k)$ is obtained in natural order (i.e., $k = 0, 1, \dots, N - 1$). On the other hand, we should indicate that it is possible to arrange the FFT algorithm such that the input is left in natural order and the resulting output DFT will occur in bit-reversed order. Furthermore, we can impose the restriction that both the input data $x(n)$ and the output DFT $X(k)$ be in natural order, and derive an FFT algorithm in which the computations are not done in place. Hence such an algorithm requires additional storage.

Another important radix-2 **FFT** algorithm, called the decimation-in-frequency algorithm, is obtained by using the divide-and-conquer approach described in Section 6.1.2 with the choice of $M = 2$ and $L = N/2$. This choice of parameters implies a column-wise storage of the input data sequence. To derive the algorithm, we begin by splitting the DFT formula into two summations, one of which involves the sum over the first $N/2$ data points and the second sum involves the last $N/2$ data points. **Thus** we obtain

$$\begin{aligned} X(k) &= \sum_{n=0}^{(N/2)-1} x(n) W_N^{kn} + \sum_{n=N/2}^{N-1} x(n) W_N^{kn} \\ &= \sum_{n=0}^{(N/2)-1} x(n) W_N^{kn} + W_N^{Nk/2} \sum_{n=0}^{(N/2)-1} x\left(n + \frac{N}{2}\right) W_N^{kn} \end{aligned} \quad (6.1.33)$$

Since $W_N^{kN/2} = (-1)^k$, the expression (6.1.33) can be rewritten as

$$X(k) = \sum_{n=0}^{(N/2)-1} \left[x(n) + (-1)^k x\left(n + \frac{N}{2}\right) \right] W_N^{kn} \quad (6.1.34)$$



$(n_2 n_1 n_0)$	$\rightarrow$	$(n_0 n_2 n_1)$	$\rightarrow$	$(n_0 n_1 n_2)$
(0 0 0)	$\rightarrow$	(0 0 0)	$\rightarrow$	(0 0 0)
(0 0 1)	$\rightarrow$	(1 0 0)	$\rightarrow$	(1 0 0)
(0 1 0)	$\rightarrow$	(0 0 1)	$\rightarrow$	(0 1 0)
(0 1 1)	$\rightarrow$	(1 0 1)	$\rightarrow$	(1 1 0)
(1 0 0)	$\rightarrow$	(0 1 0)	$\rightarrow$	(0 0 1)
(1 0 1)	$\rightarrow$	(1 1 0)	$\rightarrow$	(1 0 1)
(1 1 0)	$\rightarrow$	(0 1 1)	$\rightarrow$	(0 1 1)
(1 1 1)	$\rightarrow$	(1 1 1)	$\rightarrow$	(1 1 1)

(b)

Figure 6.8 Shuffling of the data and bit reversal.

Now, let us split (decimate) $X(k)$ into the even- and odd-numbered samples. Thus we obtain

$$X(2k) = \sum_{n=0}^{(N/2)-1} \left[x(n) + x\left(n + \frac{N}{2}\right) \right] W_{N/2}^{kn} \quad k = 0, 1, \dots, \frac{N}{2} - 1 \quad (6.1.35)$$

and

$$X(2k+1) = \sum_{n=0}^{(N/2)-1} \left\{ \left[x(n) - x\left(n + \frac{N}{2}\right) \right] W_N^{kn} \right\} W_{N/2}^{kn} \quad k = 0, 1, \dots, \frac{N}{2} - 1 \quad (6.1.36)$$

where we have used the fact that $W_N^2 = W_{N/2}$.

If we define the $N/2$ -point sequences $g_1(n)$ and $g_2(n)$ as

$$\begin{aligned} g_1(n) &= x(n) + x\left(n + \frac{N}{2}\right) \\ g_2(n) &= \left[x(n) - x\left(n + \frac{N}{2}\right) \right] W_N^n \quad n = 0, 1, 2, \dots, \frac{N}{2} - 1 \end{aligned} \quad (6.1.37)$$

then

$$\begin{aligned} X(2k) &= \sum_{n=0}^{(N/2)-1} g_1(n) W_{N/2}^{kn} \\ X(2k+1) &= \sum_{n=0}^{(N/2)-1} g_2(n) W_{N/2}^{kn} \end{aligned} \quad (6.1.38)$$

The computation of the sequences $g_1(n)$ and $g_2(n)$ according to (6.1.37) and the subsequent use of these sequences to compute the $N/2$ -point DFTs are depicted in Fig. 6.9. We observe that the basic computation in this figure involves the butterfly operation illustrated in Fig. 6.10.

This computational procedure can be repeated through decimation of the $N/2$ -point DFTs, $X(2k)$ and $X(2k+1)$. The entire process involves $v = \log_2 N$

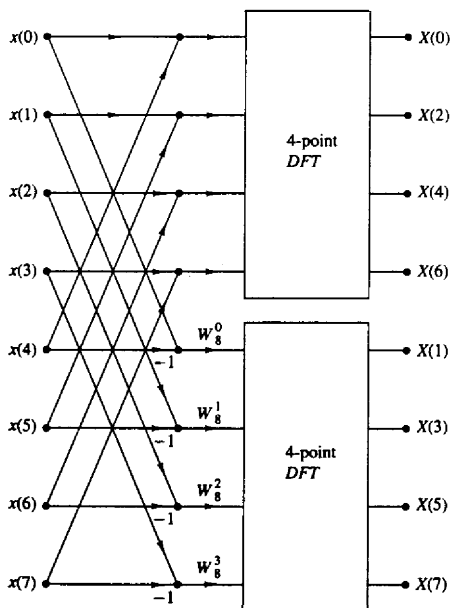


Figure 6.9 First stage of the decimation-in-frequency FFT algorithm.